VACF and Dynamical Analysis Architecture Manual

Physical Theory, Numerical Architecture, and Staged Implementation Plan for mdstats

mdstats

2026-07-30 (MLFF caller boundary; numerical architecture unchanged)

Contents

1 Purpose and status

This architecture manual records the physical theory, numerical contracts, and staged implementation plan for VACF, displacement, and collective-current analyses in `mdstats`. It is both a scientific specification and a release sequence: a stage is not complete until the implementation, focused tests, public documentation, and result provenance agree with this manual.

The VACF/spectral/Green-Kubo branch is implemented through **V5**:

1. positive-lag spectral kernels;

2. `compute_vacf_spectrum()`;
3. cumulative Green-Kubo quadrature;
4. VDOS normalization by the discrete spectral-bin measure;
5. spectrum plotting and diffusion diagnostics;
6. VACF-to-MSD reconstruction;
7. shared velocity preparation; and
8. the direct Welch velocity-spectrum estimator.

A manual-and-code audit of `mdstats 0.19.79a0` identified four contract defects at the implemented boundary: ambiguous lower-dimensional scalar divisors, incomplete MSD/VACF provenance comparison, shallow result immutability, and inconsistent validation/plateau-grid rules.

Release `mdstats 0.19.80a0` completes **H0 - dynamics contract hardening**:

- `AnalysisSubspace` makes the physical projection explicit and derives its rank;
- `DynamicsInputSignature` identifies the exact trajectory, frame slice, measured atoms, drift-reference atoms, coordinate semantics, velocity source, and projection;
- public VACF/dynamics result arrays and metadata are deeply immutable;
- strict shared validators distinguish integer and boolean controls; and
- the current arithmetic-mean plateau estimator admits only uniformly spaced selected samples.

Full three-dimensional scalar and single-Cartesian-component numerical results are preserved. Ambiguous scalar `dimensions=1/2` calls now fail unless an explicit corresponding subspace is supplied.

Release `mdstats 0.19.81a0` completes **D0 - shared displacement preparation and blocked displacement iteration**:

- `DisplacementInputBundle` resolves coordinates, reference cell, drift, selection, subspace, and signature exactly once;
- `DisplacementBlockPlan` bounds both origin and atom work under a deterministic memory target;
- `iter_displacement_blocks()` emits immutable lag-major/origin-major/atom-major samples; and
- the direct time-origin-averaged MSD consumes D0 while the FFT backend remains an independent numerical cross-check.

Release `mdstats 0.19.82a0` completes **D1 - the radial self van Hove function**:

- `compute_self_van_hove()` consumes the D0 prepared bundle and block iterator;
- one-, two-, and three-dimensional projected shell measures are explicit;
- finite user support retains exact overflow counts and probabilities rather than renormalizing captured samples;
- automatic support uses a deterministic maximum-radius prepass;
- an unbinned direct second moment provides the MSD cross-check; and
- `SelfVanHoveResult` is deeply immutable and carries the complete D0 signature.

Release `mdstats 0.19.83a0` completes **D2 - the non-Gaussian displacement parameter**:

- `compute_non_gaussian_parameter()` consumes the D0 prepared bundle and block iterator in a single pass;
- projected second and fourth moments use the same explicit one-, two-, or three-dimensional subspace;
- the rank-dependent prefactor is therefore inseparable from the projected displacement norm;
- every exact-zero second-moment lag stores `alpha2=NaN` with an explicit immutable undefined mask;
- moments cross-check directly against D1 and direct MSD; and
- `NonGaussianResult` is deeply immutable and carries the complete D0 signature.

Release `mdstats 0.19.84a0` completes **D3 - the self-intermediate scattering function**:

- `compute_self_intermediate_scattering()` consumes the D0 prepared bundle and block iterator directly;
- isotropic magnitude mode uses the dimension-correct `cos`, `J0`, or spherical-`j0` angular kernel for rank one, two, or three;
- explicit-vector mode returns the complex characteristic function and rejects vector components outside the selected physical subspace;
- zero lag and zero q are represented exactly by one;
- q ordering and duplicates are retained, while private q chunking bounds transient work; and

- `SelfIntermediateScatteringResult` is deeply immutable and carries original and projected q inputs plus the complete D0 signature.

Release `mdstats 0.19.85a0` completes **C0-C1 - collective charge-current contract closure and ordered current correlation**:

- `compute_charge_current()` resolves exactly one per-atom or exact-symbol charge source, rejects non-neutral systems, and stores total and optional exact-group currents in e angstrom/ps;
- zero-charge atoms are excluded from the current-carrying population while the full resolved charge array remains explicit;
- named groups form a disjoint exhaustive partition of the current-carrying atoms, so their currents sum exactly to the total current;
- fixed versus variable cell matrices, instantaneous volumes, periodic axes, drift provenance, and the complete dynamics signature are retained;
- `compute_current_correlation()` stores total scalar, Cartesian, and optional tensor correlations plus the full ordered positive-lag group-pair matrix;
- direct and zero-padded FFT backends share the package positive-lag correlation convention and preserve the nonsymmetric $C_{ab}(t)$ versus $C_{ba}(t)$ distinction; and
- `ChargeCurrentResult` and `CurrentCorrelationResult` are deeply immutable and validate exact group-sum and tensor-trace identities.

Release `mdstats 0.19.86a0` completes **C2 - ionic conductivity integration, explicit plateau estimation, and Nernst-Einstein comparison**:

- `integrate_ionic_conductivity()` requires fixed full-cell-matrix provenance and full three-dimensional periodicity, performs cumulative trapezoidal integration, and converts total plus ordered group-pair correlations to SI;
- `IonicConductivityResult` retains the sampled correlations, cumulative integrals, exact thermodynamic state, charge/group identity, and complete dynamics signature with constructor-level quadrature and unit checks;
- `estimate_ionic_conductivity_plateau()` applies only an explicit uniformly sampled interval and records slope, residual, span, endpoint drift, and optional stability diagnostics without claiming an independent-sample error;
- `compute_nernst_einstein_comparison()` derives populations and uniform group charges from current provenance and requires compatible full-3D species diffusion signatures; and
- both directional conductivity ratios are reported with explicit undefined flags rather than a package-imposed universal Haven-ratio convention.

The next dynamics stage is therefore **S1 - topology-coupled residence and hopping analysis**, after the required site representation is stable.

H0, D0-D3, C0-C2 are implemented and regression-tested.

The plan remains divided into independently testable implementation units. Internal numerical helpers receive specifications and direct oracle tests just like public functions. Borrowed mathematical or physical methods are cited; package-specific API, provenance, blocking, and validation decisions are identified as `mdstats` design.

2 Theoretical background

2.1 Microscopic motion and time-correlation functions

Molecular dynamics produces phase-space trajectories. For atom i, the velocity is $\mathbf{v}_i(t)$. After an explicitly chosen drift subtraction, the weighted self velocity-autocorrelation tensor is

$$C_{\alpha\beta}(t) = \frac{1}{W}\sum_i w_i \left\langle v_{i\alpha}(t_0) v_{i\beta}(t_0+t)\right\rangle_{t_0}, \qquad W = \sum_i w_i.$$

The scalar VACF is the tensor trace,

$$C_v(t) = \sum_{\alpha=1}^{d} C_{\alpha\alpha}(t) = \frac{1}{W} \sum_i w_i \left\langle \mathbf{v}_i(t_0) \cdot \mathbf{v}_i(t_0 + t) \right\rangle_{t_0}.$$

The average over time origins assumes that the analyzed production trajectory is sufficiently stationary for time translation to be meaningful. Drift removal, atom selection, weights, and trajectory interval are therefore part of the scientific definition, not merely preprocessing choices.

At $t = 0$, the VACF measures mean squared velocity. For uniform per-particle weights it describes self motion. For mass weights it is proportional to kinetic energy and is useful for vibrational spectral interpretation, but it no longer has the normalization required for a single-particle diffusion coefficient.

2.2 The displacement-correlation identity

The displacement over lag t is

$$\Delta\mathbf{r}_i(t; t_0) = \int_0^t \mathbf{v}_i(t_0 + \tau)\, d\tau.$$

For stationary dynamics, expanding the squared displacement gives

$$\mathrm{MSD}(t) = 2 \int_0^t (t - \tau) C_v(\tau)\, d\tau.$$

Equivalently,

$$\frac{d}{dt}\,\mathrm{MSD}(t) = 2 \int_0^t C_v(\tau)\, d\tau, \qquad \frac{d^2}{dt^2}\,\mathrm{MSD}(t) = 2C_v(t).$$

This identity links the displacement and velocity descriptions of the same self dynamics. It explains the short-time ballistic law when $C_v(t) \approx C_v(0)$, and the long-time linear MSD when the VACF integral reaches a finite plateau. Einstein's displacement relation [14], the Green-Kubo framework [5, 6], and Helfand's integrated-flux formulation [15] are complementary representations of transport.

For isotropic diffusion in d dimensions,

$$D = \frac{1}{d} \int_0^\infty C_v(t)\, dt = \lim_{t\to\infty} \frac{\mathrm{MSD}(t)}{2dt}.$$

For one Cartesian component,

$$D_\alpha = \int_0^\infty C_{\alpha\alpha}(t)\, dt.$$

The finite-trajectory running integral is a diagnostic curve, not automatically a converged transport coefficient. A plateau interval, tail behavior, and sampling uncertainty must be assessed explicitly.

2.3 Analysis subspaces and dimensionality

A dimensional divisor is meaningful only after the analyzed physical subspace has been defined. Let

$$B \in \mathbb{R}^{d\times 3}, \qquad BB^{\mathsf{T}} = I_d, \qquad d \in \{1, 2, 3\},$$

be an orthonormal row basis for the selected subspace. Projected velocities and displacements are

$$\mathbf{u}_i = B\mathbf{v}_i, \qquad \Delta\mathbf{s}_i = B\Delta\mathbf{r}_i.$$

The scalar VACF in that subspace is

$$C_B(t) = \mathrm{tr}\left[B\, C(t)\, B^{\mathsf{T}}\right],$$

and the corresponding scalar displacement moment is

$$M_B(t) = \left\langle \|B\Delta\mathbf{r}(t)\|^2 \right\rangle.$$

Only then are the isotropic subspace relations

$$D_B = \frac{1}{d}\int_0^\infty C_B(t)\, dt, \qquad D_B = \lim_{t\to\infty} \frac{M_B(t)}{2dt}$$

well defined. Dividing the full three-dimensional scalar correlation by an independently requested value of d is forbidden.

Canonical axis subsets are represented by rows of the Cartesian identity, for example $B = (\hat{x}, \hat{y})$ for the xy plane. A rotated basis is accepted only when the source result stores the full second-rank tensor needed to form the projection. The subspace rank is derived from B; users do not supply a second, independent `dimensions` value.

The orthonormal-projection formulation is standard linear algebra. The shared subspace resolver, exact metadata representation, compatibility policy, and fallback rules are `mdstats` design.

2.4 Frequency-domain representation

For a stationary process, the Wiener-Khinchin relation [1, 2] connects the correlation tensor to a spectral-density tensor,

$$S_{\alpha\beta}(f) = \int_{-\infty}^{\infty} C_{\alpha\beta}(t) e^{-i2\pi ft}\, dt.$$

The tensor satisfies Hermitian symmetry,

$$S_{\alpha\beta}(f) = S_{\beta\alpha}(f)^*,$$

and the scalar velocity spectrum is its trace. The spectral area recovers the zero-lag correlation under the chosen one- or two-sided convention.

Rahman's molecular-dynamics analysis of liquid argon [4] is an early example of using velocity correlations and their frequency content to characterize atomic motion. In a nearly harmonic crystal, a mass-weighted and appropriately normalized velocity spectrum can be interpreted as a finite-temperature vibrational density of states. In an anharmonic solid it includes frequency shifts and broadening. In a liquid it contains translational, cage, collision, and diffusive motion and should not be called a literal phonon DOS without qualification.

A velocity spectrum is not generally an infrared or Raman spectrum. Infrared intensity requires a dipole or charge-current correlation; Raman intensity requires a polarizability correlation. VACF spectra identify frequencies and motion-weighted spectral intensity, not optical selection rules.

2.5 Finite-record spectral estimators

A stored trajectory supplies a finite sampled signal. Two complementary spectral routes are retained.

Correlation transform. The positive-lag VACF is extended to a two-sided Hermitian sequence and transformed. The reported all-origin estimator

$$\hat{C}_{\mathrm{reported}}(k) = \frac{1}{T-k} \sum_{n=0}^{T-k-1} v_n v_{n+k}$$

preserves the displayed correlation. Multiplying by $(T-k)/T$ gives the biased form associated with the ordinary periodogram. Truncation and noisy long lags can produce negative lobes in the direct transform; those lobes are diagnostics and must not be silently erased.

Welch estimation. Welch's method [3] partitions the velocity signal into overlapping windowed segments, forms modified periodograms, and averages them. It lowers estimator variance at the cost of frequency resolution. Window leakage and main-lobe tradeoffs follow the classical analysis of Harris [12].

The sampling interval Δt imposes the Nyquist frequency

$$f_{\mathrm{N}} = \frac{1}{2\Delta t},$$

while the useful resolution is controlled by the trajectory or segment duration, not by zero padding. Zero padding refines the displayed frequency grid but adds no physical information.

2.6 Displacement distributions beyond the MSD

The MSD keeps only the second moment of displacement. The self van Hove function [7]

$$G_s(\mathbf{r}, t) = \frac{1}{N} \left\langle \sum_i \delta(\mathbf{r} - [\mathbf{r}_i(t_0 + t) - \mathbf{r}_i(t_0)]) \right\rangle_{t_0}$$

retains the full displacement distribution. It can distinguish localized rattling, broad diffusion, and discrete hopping populations that share a similar MSD.

In three dimensions, the non-Gaussian parameter

$$\alpha_2(t) = \frac{3\langle r^4(t)\rangle}{5\langle r^2(t)\rangle^2} - 1$$

measures departure from an isotropic Gaussian displacement distribution. The self-intermediate scattering function

$$F_s(\mathbf{q}, t) = \left\langle \frac{1}{N} \sum_i \exp[i\mathbf{q} \cdot (\mathbf{r}_i(t_0 + t) - \mathbf{r}_i(t_0))] \right\rangle_{t_0}$$

is the spatial Fourier transform of G_s and measures relaxation at the length scale $2\pi/|\mathbf{q}|$. These observables belong to the van Hove and scattering framework [7-9].

2.7 Collective currents and ionic conductivity

Self VACFs contain only $i = i$ correlations. Ionic conductivity is a collective transport property. For charges q_i, define the total charge current

$$\mathbf{J}_q(t) = \sum_i q_i \mathbf{v}_i(t).$$

The current autocorrelation expands into

$$\left\langle \mathbf{J}_q(0) \cdot \mathbf{J}_q(t) \right\rangle = \sum_i q_i^2 \left\langle \mathbf{v}_i(0) \cdot \mathbf{v}_i(t) \right\rangle + \sum_{i \neq j} q_i q_j \left\langle \mathbf{v}_i(0) \cdot \mathbf{v}_j(t) \right\rangle .$$

The distinct-particle terms contain correlated cation-cation, anion-anion, and cation-anion motion and cannot be reconstructed from self diffusion alone. In an isotropic system, the Green-Kubo conductivity is

$$\sigma = \frac{1}{3Vk_{\mathrm{B}}T} \int_0^\infty \left\langle \mathbf{J}_q(0) \cdot \mathbf{J}_q(t) \right\rangle dt,$$

with unit conversion handled explicitly by the implementation. The Nernst-Einstein estimate omits distinct-particle correlations; comparison with the collective result diagnoses correlation effects.

2.8 Physical conditions for trustworthy results

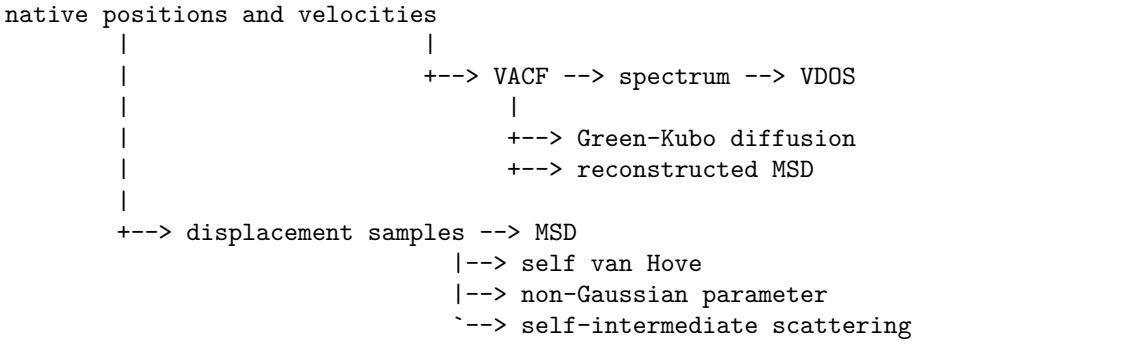
The dynamical branch is only as reliable as its trajectory semantics. The main conditions are:

- velocities must be native or otherwise accurate at the sampling times;
- the saved timestep must resolve the highest frequencies of interest;
- center-of-mass or center-of-geometry drift treatment must match the scientific question;
- the analyzed interval should be stationary and free of equilibration transients;
- diffusion integrals require a defensible plateau or tail assessment;
- mass weighting is appropriate for vibrational spectra but not for ordinary per-particle self diffusion;
- finite-record VACF and MSD estimators need not agree exactly even when both are implemented correctly.

The architecture therefore separates raw estimators, derived transforms, normalization, convergence diagnostics, and plotting. Each result keeps enough provenance to reconstruct what physical correlation was actually computed.

2.9 The dynamical-analysis architecture

The theoretical dependency structure is



The staged plan below implements these branches one function at a time. The sequence remains a programming and validation roadmap, while this section states the physical relations that make the modules one coherent architecture.

3 Terminology and scope

The phrase **phonon spectral density** is not used as the general API name. The primary computed quantity is a **velocity power spectrum**. A normalized, properly weighted spectrum may be interpreted as a finite-temperature VDOS. The name **phonon DOS** is reserved for crystalline or nearly harmonic solids where that interpretation is defensible.

A velocity spectrum is not an infrared or Raman spectrum:

- infrared intensity requires a dipole-moment or charge-current correlation;
- Raman intensity requires a polarizability correlation;
- a VACF-derived spectrum identifies frequencies and spectral weight in atomic motion, but not general optical transition intensities.

This roadmap does not modify the foundational VACF estimator. The existing `VACFResult` remains the source object for correlation-domain transforms and Green-Kubo integration.

4 Provenance convention

Every stage below labels its theoretical provenance using three categories.

Borrowed theory A published mathematical or physical method supplies the defining relation or estimator. The function specification and nearby code comments must cite the source.

Standard machinery Elementary numerical operations such as a discrete Fourier transform, trapezoidal integration, histogram accumulation, or unit conversion. These are described but normally do not require historical attribution.

mdstats design API boundaries, immutable result schemas, reversible normalization, metadata, error policies, estimator separation, deterministic ordering, and test architecture developed for this package.

A source must not be credited for mdstats-specific extensions that it does not contain.

5 Shared dynamics semantic signature

Cross-module comparisons require equality of the physical input semantics, not merely equality of filenames or a drift-mode label. H0 introduces one internal, deeply immutable signature shared by velocity- and displacement-derived results:

```
@dataclass(frozen=True, slots=True)
class DynamicsInputSignature:
    source_format: str | None
    source_files: tuple[str, ...]
    trajectory_fingerprint: str

    frame_indices: tuple[int, ...] | None
    frame_times_ps: NDArray[np.float64]
    n_frames: int
    sample_spacing_ps: float | None

    atom_indices: NDArray[np.int64]
    coordinate_mode: str
    reference_cell_mode: str | None
    reference_cell: NDArray[np.float64] | None
```

```
    drift_mode: str | None
    drift_atom_indices: NDArray[np.int64] | None
    velocity_source: str | None

    projection_basis: NDArray[np.float64]
    projection_labels: tuple[str, ...] | None
```

The exact implementation may store compact digests in addition to, or instead of, duplicated large arrays, but equality must cover the information listed above. `trajectory_fingerprint` is a deterministic digest of the actual analyzed frame sequence, atomic numbers and masses, periodic flags, cells and origins, fractional positions, times, and velocities when present. Equal source filenames alone do not prove equal slices.

Compatibility rules are fail-closed:

- measured atoms must match in canonical order;
- frame sequence, times, and sample spacing must match;
- coordinate and reference-cell conventions must match;
- drift mode and the exact drift-reference atom indices must match;
- velocity provenance must match for velocity-based comparisons;
- the same physical projection subspace must be used; and
- observable-specific exceptions must be explicit in the comparison function, never inferred from absent metadata.

Legacy result constructors remain loadable for isolated compatibility, but a result that lacks a complete signature is rejected by cross-module scientific comparison. New computed results always carry the complete signature.

6 Deep immutability and validation boundary

`@dataclass(frozen=True)` prevents field rebinding but does not make NumPy arrays or nested metadata immutable. Every public result must therefore:

1. copy or take exclusive ownership of stored arrays;
2. normalize their dtypes and shapes;
3. set `writeable=False` on every stored array;
4. recursively freeze metadata mappings, sequences, and nested arrays; and
5. validate identities before exposing the object.

Internal ephemeral work arrays do not need this treatment. Public result arrays, input signatures, diagnostics, and metadata do.

All positive-integer and boolean options use shared strict validators. Python booleans are rejected where an integer is required, and boolean switches reject integer substitutes. This is an `mdstats` API rule rather than a borrowed numerical method.

7 Existing foundation

The roadmap builds on the current package contracts:

```
AtomisticFrameCollection
    |-- uniform trajectory times in ps
    |-- Cartesian velocities in A/ps
    |-- unwrapped positions when displacement analysis is requested
    |-- fixed atom identity, species, and masses

compute_vacf(...) -> VACFResult
    |-- positive-lag raw weighted self correlation
    |-- scalar, Cartesian, optional tensor, optional per-atom forms
    |-- exact lag times and origin counts
    |-- uniform, mass, or explicit nonnegative atom weights
```

```
compute_msd(...) -> MSDResult
    |-- time-origin displacement statistics
    |-- species and atom selection
    |-- direct and FFT estimators where applicable
```

The first spectrum function must consume `VACFResult` directly. It must not recompute a VACF internally.

8 Locked numerical decisions

The following decisions are normative for the first implementation. Changing one of them later requires a documented API or result-metadata change.

Topic	Final first-generation decision	Reason
Canonical frequency	cycles/ps, numerically THz	Avoid hidden factors of 2π; derive rad/ps, cm^{-1}, and meV.
VACF transform	Hermitian two-sided reconstruction followed by `scipy.fft.rfft`	One kernel handles scalar, Cartesian, tensor, and per-atom correlations.
DCT alternative	Not used in the production kernel	A scalar DCT is convenient, but a full tensor also requires sine information for antisymmetric cross terms.
Spectral sidedness	one-sided density	Gives a direct nonnegative-frequency representation and a clear spectral-area identity.
Correlation estimator	explicit `reported` or `biased` option	The displayed all-origin VACF and the periodogram-compatible lag weighting are not silently conflated.
Lag taper	none by default; centered half-windows only	An ordinary full Hann array would incorrectly erase $C(0)$.
Zero padding	`scipy.fft.next_fast_len`, with explicit lower bound	Refines the sampled frequency grid without claiming extra physical resolution.
Direct velocity PSD	Welch averaging with Hann/50% overlap as a recommended preset	Reduces estimator variance with an explicit resolution tradeoff.
Welch detrending	`none` by default	Segment-mean removal can suppress genuine low-frequency transport and must be explicit.
Green-Kubo integral	cumulative composite trapezoid	Stable, transparent, length-preserving, and appropriate for sampled noisy correlations.
VDOS area	uniform FFT-bin sum, not trapezoidal quadrature	Preserves the discrete one-sided FFT identity and zero-padding invariance.
Negative spectrum policy	preserve by default; clip roundoff only when requested	A transformed reported VACF may have meaningful negative lobes from noise or truncation.
Analysis dimensionality	derive d from an explicit orthonormal subspace	Prevents full 3D scalar data from being divided by an unrelated 1D/2D integer.
Cross-module compatibility	compare `DynamicsInputSignature`	Filenames and drift-mode names are insufficient provenance.

Topic	Final first-generation decision	Reason
Public result immutability	recursively frozen metadata and read-only owned arrays	Frozen dataclasses alone do not prevent mutation.
Plateau sample measure	require a uniform selected lag grid for arithmetic averaging	Avoids silently overweighting densely sampled regions.
Charge neutrality	neutral periodic system required by default	A uniform drift changes current by $Q_{\mathrm{tot}}\mathbf{v}_{\mathrm{drift}}$ when net charge is nonzero.
Irregular sampling	rejected	Resampling and nonuniform spectral estimators require a separate stage.

The PSD/autocorrelation relation is borrowed from Wiener-Khinchin theory [1, 2]. Welch segment averaging is borrowed from Welch [3]. Window tradeoffs follow the established harmonic-analysis treatment of Harris [12]. The specific estimator switches, tensor reconstruction, metadata, and error policies are mdstats designs.

9 Numerical architecture and module reuse

9.1 Separation of numerical contracts

The existing internal module `_fft.py` computes **linear correlations** from signals. The new spectral branch computes **spectra from correlations or signals**. These are inverse-looking operations but they have different padding, scaling, and statistical contracts and must not be forced through one ambiguous helper.

```
_fft.py
    signal arrays -> linear positive-lag correlation sums

_spectral.py
    positive-lag correlation -> one-sided spectral density
    windowed signal segments -> one-sided periodograms

_quadrature.py
    sampled correlation -> cumulative integral on the original lag grid
```

SciPy supplies the FFT, windows, constants, and cumulative trapezoid routines; mdstats owns the scientific normalization and result semantics. SciPy should be cited as a direct software dependency [13].

9.2 Reuse matrix

`_fft.linear_fft_length()` Do not call for spectra. Its lower bound is defined for linear-correlation construction rather than spectral sampling.

`_fft.positive_lag_pair_counts()` Reuse the concept and normally consume `VACFResult.n_origins`. It is needed for explicit biased lag weighting.

`_fft.positive_lag_correlation_from_spectrum()` Do not call. It performs spectrum-to-correlation accumulation, which is the opposite numerical contract.

`_fft.make_atom_fft_plan()` Reuse the architecture, not the unchanged code. Spectral and displacement blocking have different work-array shapes and padding rules.

The `_fft.py` atom-blocking pattern Reuse deterministic block ordering and bounded-memory planning.

`vacf.py` results and metadata Reuse directly in VS1 and GK1; never recompute the VACF inside a consumer.

VACF selection, weighting, and drift helpers Reuse through `_velocity_common.py` so direct Welch and current construction cannot diverge from VACF semantics.

`selection.py` and `collection.py` Reuse canonical atom selection, time, velocity, mass, source, and provenance validation.

`msd.py` Refactor only the direct time-averaged path onto D0 after H0. Preserve the FFT square-expansion backend as an independent implementation and regression oracle.

`_dynamics_common.py` New H0 internal boundary for subspace resolution, semantic signatures, deep-freeze helpers, and strict validators.

`_displacement_common.py` New D0 internal boundary for coordinate preparation and deterministic lag/origin/atom-block iteration.

`io/units.py` Do not extend with spectral axes. Keep a small analysis-level spectral-unit helper using `scipy.constants`.

Existing plotting modules Reuse the result-object interface pattern. Plotting returns `Axes` and never renormalizes silently.

9.3 Intended package layout

```
mdstats/analysis/
    _fft.py                         # existing linear-correlation primitives
    _spectral.py                    # spectral transforms and PSD scaling
    _quadrature.py                  # cumulative sampled-data integration
    _velocity_common.py             # shared velocity selection/weights/drift
    _spectral_units.py              # THz/rad-ps/cm^-1/meV conversion
    _dynamics_common.py             # H0 subspace/signature/freeze/validation
    _displacement_common.py         # D0 preparation and blocked iterator

    vacf.py                         # foundational self-VACF estimator
    velocity_spectrum.py            # VS1, VS2, VS3 and result types
    vacf_transport.py               # GK1 and GK4
    diffusion.py                    # GK2 and GK3
    displacement_dynamics.py        # DH1 and DH2
    intermediate_scattering.py      # DH3 and later collective forms
    current_correlation.py          # C0, CC1, and CC2 (implemented)
    ionic_conductivity.py           # CC3, conductivity plateau, and CC4 (implemented C2)

mdstats/plotting/
    velocity_spectrum.py
    vacf_transport.py
    displacement_dynamics.py
    current_correlation.py
```

Public analysis functions remain independent of plotting. Internal helpers do not become public exports merely because they have their own tests.

10 Internal numerical implementation units

These helpers are implemented before the first public function that depends on them.

10.1 N1.1 - `resolve_spectrum_fft_length`

```python
def resolve_spectrum_fft_length(
    n_positive_lags: int,
    *,
    zero_pad_to: int | None,
) -> int:
    ...
```

Requirements:

- require `n_positive_lags >= 1`;
- use a base length of at least $2L - 1$ for L stored nonnegative lags;
- honor a larger user lower bound;
- call `scipy.fft.next_fast_len` for the resolved length;
- never claim that padding improves physical resolution.

Tests compare exact lower bounds, explicit padding requests, and frequency-grid spacing.

10.2 N1.2 - `resolve_lag_window`

```
def resolve_lag_window(
    window: LagWindowInput | None,
    n_lags: int,
) -> tuple[NDArray[np.float64], dict[str, Any]]:
    ...
```

Built-ins initially support:

- `None` / rectangular truncation;
- `"half_hann"`;
- `("half_tukey", alpha)`;
- a custom finite array of length `n_lags` with $w_0 = 1$.

For the half-Hann taper,

$$w_k = \frac{1}{2}\left[1 + \cos\left(\frac{\pi k}{L-1}\right)\right], \qquad k = 0, \ldots, L-1.$$

Thus $w_0 = 1$ and the last measured lag is tapered to zero. The window design is borrowed signal-processing machinery [12]; the positive-lag API and validation rules are mdstats-specific.

10.3 N1.3 - `reconstruct_two_sided_correlation`

```
def reconstruct_two_sided_correlation(
    positive_lag: NDArray[np.float64],
    *,
    n_fft: int,
    tensor_axes: tuple[int, int] | None = None,
) -> NDArray[np.float64]:
    ...
```

For a tensor VACF,

$$C_{\alpha\beta}(-k\Delta t) = C_{\beta\alpha}(k\Delta t).$$

The helper places positive lags at the beginning of the periodic work array, negative lags at the end, and zeros in the unused middle. It must preserve the tensor-transpose relation exactly enough that the transformed spectrum is Hermitian within floating-point tolerance.

The stationarity symmetry is standard correlation theory. Array layout, generalized axis handling, and deterministic reconstruction are mdstats designs.

10.4 N1.4 - `one_sided_density_scale`

```
def one_sided_density_scale(n_fft: int) -> NDArray[np.float64]:
    ...
```

The scale vector leaves DC unchanged, doubles positive interior bins, and leaves the Nyquist bin unchanged when `n_fft` is even. It must handle odd and even transform lengths correctly.

10.5 N1.5 - `transform_positive_lag_correlation`

```
def transform_positive_lag_correlation(
    correlation: NDArray[np.float64],
    *,
```

```
    dt_ps: float,
    n_fft: int,
    tensor_axes: tuple[int, int] | None = None,
) -> tuple[NDArray[np.float64], NDArray[np.complex128]]:
    ...
```

Algorithm:

1. reconstruct the real two-sided sequence;
2. call `scipy.fft.rfft` along the lag axis;
3. multiply by Δt to approximate the continuous transform;
4. apply the one-sided density scale;
5. return `scipy.fft.rfftfreq(n_fft, d=dt_ps)` and the spectrum.

A direct $O(LF)$ Fourier sum is implemented only in tests as an independent oracle.

10.6 N1.6 - `spectral_bin_integral`

Implementation status: completed in `mdstats 0.19.2a0`. See `docs/specs/analysis/_spectral_spec.md`.

```
def spectral_bin_integral(
    spectrum: NDArray[np.float64],
    frequencies_thz: NDArray[np.float64],
    *,
    axis: int = 0,
) -> NDArray[np.float64]:
    ...
```

For the uniform FFT grid,

$$I = \Delta f \sum_m P_+(f_m).$$

This is a discrete bin measure, not a trapezoidal approximation to an interpolated curve. It is used by VDOS normalization and Parseval-style tests.

10.7 N2.1 - `cumulative_trapezoid_zero`

Implementation status: completed in `mdstats 0.19.1a0`. See `docs/specs/analysis/_quadrature_spec.md`.

```
def cumulative_trapezoid_zero(
    values: NDArray[np.float64],
    coordinates: NDArray[np.float64],
    *,
    axis: int = 0,
) -> NDArray[np.float64]:
    ...
```

This wrapper validates finite monotonic coordinates, converts the computation to `float64`, and delegates to `scipy.integrate.cumulative_trapezoid(..., initial=0.0)`. The composite trapezoidal rule is standard numerical-analysis machinery; the validation and length-preserving package contract are mdstats designs.

10.8 VC0 - shared velocity-input preparation

Implementation status: completed in `mdstats 0.19.6a0`. See `docs/specs/analysis/_velocity_common_spec.md`.

VC0 extracts the existing VACF trajectory validation, measured/drift selection, atom weighting, framewise drift construction, and per-atom output mapping into `_velocity_common.py`. The private `VelocityInputBundle` retains the canonical velocity field without a large copy and carries resolved small arrays plus the optional `(T, 3)` drift velocity.

VC0 introduces no new physical or mathematical estimator. It is an mdstats refactor whose acceptance condition is numerical and behavioral parity with the existing VACF API. Physical drift removal remains separate from Welch segment detrending.

10.9 N3.1 - make_atom_spectrum_plan

Implementation status: completed in `mdstats 0.19.6a0`. See `docs/specs/analysis/_spectral_spec.md`.

```python
@dataclass(frozen=True, slots=True)
class AtomSpectrumPlan:
    n_fft: int
    n_frequency: int
    atom_block_size: int
    estimated_work_bytes: int


def make_atom_spectrum_plan(
    n_atoms: int,
    segment_length: int,
    n_fft: int,
    *,
    atom_block_size: int | None,
    memory_target_bytes: int,
) -> AtomSpectrumPlan:
    ...
```

This follows the existing `_fft.py` memory-planning pattern but uses spectral work-array sizes. It is implemented immediately before VS2.

11 Roadmap overview

```
Implemented through C1
    |
    +--> H0 dynamics contract hardening [complete]
            |-- H0.1 explicit analysis subspace
            |-- H0.2 DynamicsInputSignature
            |-- H0.3 deep result immutability
            |-- H0.4 strict validation and plateau-grid policy
            |
            +--> D0 displacement preparation + blocked iterator [complete]
            |       |--> D1 self van Hove [complete]
            |       |--> D2 non-Gaussian parameter [complete]
            |       `--> D3 self-intermediate scattering [complete]
            |
            +--> C0 charge/current contract [complete]
                    |--> C1 charge current + ordered correlations [complete]
                    `--> C2 conductivity + Nernst-Einstein comparison [next]

Stable ring/site topology
    |
    `--> S1 topology-coupled residence and hopping statistics
```

The exact one-function-at-a-time order is:

1. **N1-V5 - implemented foundation.** Orders 1-17 comprise the spectral, Green-Kubo, plotting, reconstruction, shared-velocity, and Welch units.
2. **H0.1-H0.5 - implemented in 0.19.80a0.** The shared subspace resolver, complete semantic signature, recursive result freezing, strict validation, plateau-grid enforcement, and corrected GK1/GK3 semantics are

complete.

3. **D0.1-D0.3 - implemented in 0.19.81a0.** Prepared displacement inputs, deterministic lag/origin/atom blocking, conservative memory planning, and direct time-averaged MSD migration are complete.
4. **D1 - `compute_self_van_hove` - implemented in 0.19.82a0.** D1 reuses D0 without reconstructing coordinates, drift, selections, or provenance and adds finite-support/overflow accounting plus the direct second-moment oracle.
5. **D2 - `compute_non_gaussian_parameter` - implemented in 0.19.83a0.** D2 reuses D0 projected displacement samples for direct second and fourth moments, exact zero-moment masking, and the rank-correct cumulant prefactor.
6. **D3 - `compute_self_intermediate_scattering` - implemented in 0.19.84a0.** D3 reuses D0 for direct isotropic and explicit-vector characteristic-function estimates, exact q identity, and bounded q chunking.
7. **C0 - implemented in 0.19.85a0.** Charge-source resolution, default neutrality, exact current-group partitioning, cell/volume provenance, and fail-closed comparison identity are complete.
8. **C1.1 - `compute_charge_current` - implemented in 0.19.85a0.** The function reuses the hardened velocity input layer while retaining distinct collective charge algebra.
9. **C1.2 - `compute_current_correlation` - implemented in 0.19.85a0.** Direct and `_fft.py` positive-lag paths retain intentional cross-particle and ordered group-pair terms.
10. **C2.1 - `integrate_ionic_conductivity` - implemented in 0.19.86a0.** Reuse validated cumulative trapezoidal quadrature after fixed-cell and full periodicity checks, with exact SI conversion.
11. **C2.2 - `estimate_ionic_conductivity_plateau` - implemented in 0.19.86a0.** Select only explicit uniformly sampled intervals and retain ordered group-pair means plus stability diagnostics.
12. **C2.3 - `compute_nernst_einstein_comparison` - implemented in 0.19.86a0.** Require compatible species diffusion, group charges, thermodynamic state, trajectory identity, and drift provenance.
13. **S1 and later.** Resume site residence and hopping after the topology representation is stable.

A unit is complete only when its specification, code, direct-oracle tests, provenance comments, architecture notes, and public documentation are synchronized.

12 Shared spectral result type

VS1 and VS2 return the same result class so that VDOS and plotting remain independent of the estimator.

```
@dataclass(frozen=True, slots=True)
class VelocitySpectrumResult:
    frequencies_thz: NDArray[np.float64]       # (F,)
    angular_frequencies_ps_inv: NDArray[np.float64]  # (F,)
    wavenumbers_cm_inv: NDArray[np.float64]    # (F,)
    energies_mev: NDArray[np.float64]          # (F,)

    scalar_spectrum: NDArray[np.float64]        # (F,)
    component_spectra: NDArray[np.float64]      # (F, 3)
    tensor_spectrum: NDArray[np.complex128] | None  # (F, 3, 3)

    per_atom_scalar: NDArray[np.float64] | None       # (F, M)
    per_atom_components: NDArray[np.float64] | None   # (F, M, 3)
    per_atom_indices: NDArray[np.int64] | None        # (M,)

    atom_indices: NDArray[np.int64]
    atom_weights: NDArray[np.float64]
    weight_sum: float

    estimator: Literal["vacf_transform", "welch"]
    weighting: str
    normalization: str
    correlation_weighting: str | None
    spectral_sidedness: Literal["one_sided"]
    spectral_scaling: Literal["density"]
```

```
    spectrum_units: str
    sample_spacing_ps: float
    n_samples: int
    n_fft: int
    window: str | None
    detrend: str | None

    metadata: dict[str, Any]
```

Required identities are:

- `scalar_spectrum == component_spectra.sum(axis=1)` within tolerance;
- tensor diagonal equals `component_spectra` when the tensor exists;
- the tensor spectrum is Hermitian at every frequency;
- all frequency axes are mutually consistent;
- the frequency grid starts at zero and is uniformly spaced;
- VS2 diagonal spectra are nonnegative up to roundoff;
- VS1 with `correlation_weighting="reported"` is **not** required to be nonnegative because finite noisy lag estimates and truncation can produce negative lobes;
- no result consumer may infer normalization, sidedness, or estimator from array magnitude.

For a diagonal one-sided density obtained from a complete reconstructed correlation,

$$\Delta f \sum_m P_+(f_m) \approx C(0).$$

This identity is a required test, with tolerance determined by floating-point roundoff and any explicit lag taper.

13 VS1 - VACF-to-spectrum transform

13.1 Public function

```
def compute_vacf_spectrum(
    vacf: VACFResult,
    *,
    normalization: Literal["raw", "per_weight"] = "per_weight",
    correlation_weighting: Literal["reported", "biased"] = "reported",
    window: LagWindowInput | None = None,
    zero_pad_to: int | None = None,
    sidedness: Literal["one_sided"] = "one_sided",
    negative_policy: Literal[
        "preserve",
        "clip_roundoff",
        "error",
    ] = "preserve",
    negative_tolerance: float = 1.0e-12,
) -> VelocitySpectrumResult:
    ...
```

13.2 Motive

This is the smallest useful extension of the existing VACF module. It produces frequency-domain information without rereading or reprocessing the trajectory. It is the first public function implemented after the internal spectral kernels.

13.3 Borrowed theory and mdstats contribution

For a stationary process, the power spectral density is the Fourier transform of the autocorrelation. This is the Wiener-Khinchin theorem [1, 2]. Applying velocity-correlation spectra in MD has an established history, including Rahman's liquid-argon work [4].

Borrowed:

- autocorrelation/spectral-density relation [1, 2];
- FFT and window machinery as implemented by SciPy [12, 13].

mdstats-specific:

- the positive-lag tensor reconstruction API;
- explicit `reported` versus `biased` correlation weighting;
- one-sided density and unit metadata;
- reversible raw/per-weight normalization;
- negative-value policy and invariant checks.

13.4 Correlation-estimator contract

Let the reported all-origin VACF be

$$C_{\text{reported}}(k) = \frac{1}{T-k} \sum_{n=0}^{T-k-1} v_n v_{n+k}.$$

The periodogram-compatible finite-record form is

$$C_{\text{biased}}(k) = \frac{1}{T} \sum_{n=0}^{T-k-1} v_n v_{n+k} = \frac{T-k}{T} C_{\text{reported}}(k).$$

In the general `VACFResult`, use

$$C_{\text{biased}}(k) = \frac{n_{\text{origins}}(k)}{n_{\text{origins}}(0)} C_{\text{reported}}(k).$$

The package must never apply this factor silently.

- `reported` literally transforms the stored estimator;
- `biased` applies the finite-origin triangular weighting before any optional lag taper.

13.5 Discrete transform contract

For scalar or diagonal $C(k\Delta t)$, reconstruct a real two-sided sequence. For a tensor,

$$C_{\alpha\beta}(-k\Delta t) = C_{\beta\alpha}(k\Delta t).$$

After optional lag weighting and tapering,

$$S_{\alpha\beta}(f_m) \approx \Delta t \ \text{RFFT}[g_{\alpha\beta}]_m.$$

Apply one-sided density scaling:

$$P_+(f_m) = \begin{cases} S(f_m), & m = 0, \\ 2S(f_m), & 0 < m < m_{\text{last}}, \\ S(f_m), & m = m_{\text{Nyquist}} \text{ when present.} \end{cases}$$

The canonical grid is

$$f_m = \frac{m}{N_{\mathrm{FFT}}\Delta t}.$$

The production implementation uses `rfft`, not a DCT. A direct cosine/sine sum remains the independent small-array test oracle.

13.6 Inputs and constraints

- `vacf` must contain lag zero and a contiguous uniform physical lag grid.
- `normalization="raw"` transforms stored weighted correlations.
- `normalization="per_weight"` divides by `vacf.weight_sum` before the transform.
- `window=None` means rectangular lag truncation.
- Built-in lag windows are centered at zero and satisfy $w_0 = 1$.
- A custom window must be finite, one-dimensional, length-compatible, and satisfy $w_0 = 1$ within tolerance.
- `zero_pad_to` is a lower bound on FFT length, not a resolution claim.
- `negative_policy="preserve"` is the default.
- Only tiny diagonal negatives may be clipped under `clip_roundoff`.
- Tensor off-diagonal values remain complex and are never clipped.

13.7 Algorithm

1. Validate `VACFResult`, lag spacing, and normalization metadata.
2. Select raw or per-weight correlation arrays.
3. Apply explicit `reported` or `biased` origin weighting.
4. Resolve and multiply the lag-domain window.
5. Resolve `n_fft` using N1.1.
6. Transform scalar, Cartesian, tensor, and per-atom arrays through N1.5.
7. Build all frequency axes through `_spectral_units.py` from THz.
8. Apply the requested negative-value policy only to scalar/diagonal arrays.
9. Validate trace, diagonal, Hermitian, frequency, and metadata identities.
10. Return immutable arrays and complete provenance.

13.8 Required tests

- constant correlation and exact DC behavior;
- one sinusoid and two sinusoids at on-grid and off-grid frequencies;
- direct $O(LF)$ Fourier sum agrees with N1.5;
- tensor positive-lag input produces a Hermitian spectrum;
- raw and per-weight results differ exactly by `weight_sum`;
- `biased` equals `reported * n_origins/n_origins[0]` before transformation;
- one-sided bin area reproduces the transformed zero-lag value;
- zero padding changes grid spacing but not bin-integrated weight;
- half-Hann suppresses terminal discontinuity without changing $C(0)$;
- reported noisy VACF may remain negative without clipping;
- biased exact synthetic autocorrelation is nonnegative up to roundoff;
- malformed lags, windows, weights, padding, and policies are rejected.

13.9 Acceptance condition

VS1 is complete when N1.1-N1.5 and VS1 specifications, implementations, tests, public export, changelog entry, Markdown documentation, and PDF documentation all pass. VS1 must not depend on VS2.

14 VS2 - Direct Welch velocity spectrum

Implementation status: completed in `mdstats 0.19.7a0`. See `docs/specs/analysis/velocity_spectrum_spec.md`.

14.1 Public function

```python
def compute_velocity_spectrum(
    collection: AtomisticFrameCollection,
    *,
    species: SpeciesSelection = None,
    atom_indices: ArrayLike | None = None,
    weights: WeightInput = "uniform",
    drift_mode: DriftMode | None = None,
    drift_species: SpeciesSelection = None,
    drift_atom_indices: ArrayLike | None = None,
    segment_length: int | None = None,
    overlap: float | int = 0.5,
    window: str | tuple[str, float] = "hann",
    detrend: Literal["none", "constant"] = "none",
    zero_pad_to: int | None = None,
    compute_tensor: bool = True,
    per_atom: bool = False,
    per_atom_indices: ArrayLike | None = None,
    atom_block_size: int | None = None,
    memory_target_bytes: int = 256_000_000,
) -> VelocitySpectrumResult:
    ...
```

14.2 Motive

A transformed VACF uses the full correlation estimate but can be sensitive to long-lag noise and truncation. Welch averaging supplies an independent direct spectral estimator with an explicit variance-resolution tradeoff.

14.3 Borrowed theory and mdstats contribution

The estimator follows Welch's method: split the record into overlapping segments, window each segment, compute modified periodograms, and average them [3]. Window selection and leakage tradeoffs follow standard harmonic-analysis practice [12]. FFT and window functions are supplied by SciPy [13].

mdstats adds atom selection, drift semantics, weighted self-only aggregation, Cartesian tensor output, atom blocking, immutable results, and cross-estimator checks.

14.4 Numerical contract

For segment s, atom i, component α, and atom weight q_i, transform

$$X_{si\alpha}(f) = \mathrm{RFFT}\left[w_n \sqrt{q_i}\, v_{si\alpha}(n)\right].$$

The component density is

$$P_{\alpha\alpha}(f) = \frac{1}{N_{\mathrm{seg}}} \sum_s \frac{\sum_i |X_{si\alpha}(f)|^2}{f_s \sum_n w_n^2},$$

and the tensor cross spectrum is

$$P_{\alpha\beta}(f) = \frac{1}{N_{\text{seg}}} \sum_s \frac{\sum_i X_{si\alpha}(f)^* X_{si\beta}(f)}{f_s \sum_n w_n^2}.$$

The same one-sided scale as N1.4 is then applied. The sum over atoms contains only per-atom self periodograms; no $i \neq j$ products are introduced.

14.5 Inputs and constraints

- The collection must be a time-ordered trajectory with complete velocities.
- Sampling must be uniform.
- Segment length must contain at least two samples and not exceed the record.
- Overlap must leave a positive integer advance.
- Window-energy normalization is mandatory and stored in metadata.
- `detrend="none"` is the default after explicit physical drift removal.
- `detrend="constant"` removes each segment mean and must be reported because it suppresses low-frequency content.
- The first implementation supports no irregular-time resampling.
- Per-atom spectra use N3.1 blocking to bound memory.

14.6 Algorithm

1. Reuse extracted VACF input-selection, weighting, and drift helpers.
2. Resolve deterministic segment starts and overlap.
3. Construct the segment window through `scipy.signal.get_window`.
4. Optionally subtract each segment mean.
5. Multiply atom velocities by $\sqrt{q_i}$ and the segment window.
6. Apply blocked `rfft` to atom/component arrays.
7. Accumulate self periodograms and optional component cross spectra.
8. Average over segments and divide by $f_s \sum w^2$.
9. Apply N1.4 one-sided scaling.
10. Construct the shared `VelocitySpectrumResult` and validate invariants.

14.7 Required tests

- one sinusoid peak location and density normalization;
- two-frequency signal and leakage behavior;
- white-noise spectrum flat in expectation using deterministic seeded cases;
- exact no-cross-atom contamination test;
- tensor Hermiticity and component/trace identities;
- overlap, step, and segment-count identities;
- Hann window-energy normalization through Parseval checks;
- one rectangular segment agrees with a hand periodogram;
- scalar and two-series cases agree with `scipy.signal.welch` and `csd`;
- atom blocking is invariant to block size;
- `detrend="constant"` removes DC while `none` preserves it;
- approximate agreement with VS1 `correlation_weighting="biased"` on a long stationary synthetic trajectory.

14.8 Implementation timing

VS2 follows the VS1, VDOS, plotting, and Green-Kubo core. VS1 first establishes the result schema, frequency axes, sidedness, and scaling with a smaller implementation surface.

15 VS3 - VDOS normalization

Implementation status: completed in `mdstats 0.19.2a0`. See `docs/specs/analysis/vdos_spec.md`.

15.1 Public function

```python
def compute_vdos(
    spectrum: VelocitySpectrumResult,
    *,
    normalization: Literal[
        "unit_area",
        "degrees_of_freedom",
        "none",
    ] = "unit_area",
    target_degrees_of_freedom: float | None = None,
    minimum_frequency_thz: float | None = None,
    negative_policy: Literal["error", "clip_roundoff"] = "clip_roundoff",
    negative_tolerance: float = 1.0e-12,
) -> VDOSResult:
    ...
```

15.2 Result type

```python
@dataclass(frozen=True, slots=True)
class VDOSResult:
    frequencies_thz: NDArray[np.float64]
    wavenumbers_cm_inv: NDArray[np.float64]
    energies_mev: NDArray[np.float64]

    total: NDArray[np.float64]
    components: NDArray[np.float64]
    per_atom: NDArray[np.float64] | None
    per_atom_indices: NDArray[np.int64] | None

    normalization: str
    integrated_weight_before: float
    integrated_weight_after: float
    target_weight: float | None
    source_estimator: str
    weighting: str
    metadata: dict[str, Any]
```

15.3 Motive

A velocity spectrum and a normalized VDOS are related but not identical. Keeping them separate prevents silent normalization changes and avoids calling all liquid spectra a phonon DOS.

15.4 Borrowed theory and mdstats contribution

Velocity-correlation spectra as descriptions of vibrational dynamics are established in MD, including early liquid-dynamics work by Rahman [4]. The 2PT method later uses a VACF-derived density of states for approximate liquid thermodynamics [10], but VS3 does **not** implement 2PT.

The discrete bin normalization, conservative naming policy, degrees-of-freedom requirements, and result schema are mdstats designs.

15.5 Interpretation contract

- A mass-weighted spectrum is preferred for a VDOS interpretation.
- A uniform-weight spectrum remains a valid velocity spectrum but does not silently acquire a phonon-DOS label.
- `unit_area` normalizes the retained total bin measure to one.
- `degrees_of_freedom` normalizes to an explicit target.
- The function does not infer constrained degrees of freedom unless source metadata proves them.
- Low-frequency removal is explicit and preserved in metadata.
- A spectrum with material negative bins is rejected; only roundoff-level negatives may be clipped.

15.6 Numerical integration contract

For a uniform one-sided FFT grid,

$$I = \Delta f \sum_m P_+(f_m).$$

Use N1.6. Do **not** use trapezoidal endpoint half-weights, because the stored values already represent one-sided discrete FFT bins. This choice preserves the zero-padding invariance of total spectral weight.

15.7 Algorithm

1. Validate the shared frequency, sidedness, and density metadata.
2. Apply the negative-value policy to total and projections.
3. Optionally remove bins below an explicit threshold.
4. Compute the pre-normalization weight through N1.6.
5. Resolve the requested target and one scalar normalization factor.
6. Apply exactly the same factor to total, components, and per-atom arrays.
7. Recompute and store the post-normalization bin measure.
8. Validate projection sums and immutable metadata.

15.8 Required tests

- unit-area normalization by discrete bin sum;
- explicit $3N$, $3N-3$, and arbitrary target normalization;
- projection sums reproduce the total;
- normalization invariant under zero padding;
- a materially negative spectrum is rejected;
- roundoff clipping changes only tolerance-level values;
- exact preservation under `normalization="none"`;
- trapezoidal integration is deliberately shown not to be the normative FFT-bin measure at the one-sided endpoints.

16 VP1 - Velocity-spectrum plotting

Implementation status: completed in `mdstats 0.19.3a0`. See `docs/specs/plotting/velocity_spectrum_spec.md`.

16.1 Public function

```
def plot_velocity_spectrum(
    result: VelocitySpectrumResult | VDOSResult,
    *,
    x_axis: Literal["thz", "cm^-1", "mev"] = "thz",
    projection: Literal["total", "components", "per_atom"] = "total",
    atom_indices: Sequence[int] | None = None,
```

```
    normalize_for_display: bool = False,
    ax: matplotlib.axes.Axes | None = None,
) -> tuple[matplotlib.figure.Figure, matplotlib.axes.Axes]:
    ...
```

The tuple return follows the established mdstats plotting API. Alternate horizontal coordinates reuse stored axes and do not apply a density Jacobian; the ordinate remains normalized with respect to the THz grid.

16.2 Design rules

- Never renormalize silently.
- Label the y axis from result units and normalization.
- Clearly distinguish velocity spectrum, VDOS, and phonon-DOS language.
- Allow THz, inverse-centimeter, and meV axes without recomputing data.
- Component plots use the stored Cartesian projections.
- Per-atom plots require an explicit subset or at most twelve implicit curves.
- Explicit display normalization uses one common maximum-absolute scale.
- Plotting returns `(Figure, Axes)`; file writing remains a separate user action.

16.3 Required tests

- all supported axes;
- correct labels for raw, per-weight, and VDOS data;
- component and per-atom selection;
- bounded implicit per-atom output and strict requested-index handling;
- no mutation of the result;
- no hidden display normalization unless requested;
- existing-axes reuse and public imports.

17 H0 - Dynamics contract hardening

Status: implemented in `mdstats 0.19.80a0`; D0 was completed in `0.19.81a0`.

H0 changes no physical estimator when the existing call already represents the full three-dimensional scalar or one Cartesian component. It repairs ambiguous one-/two-dimensional scalar behavior and makes cross-module provenance and immutability enforceable.

17.1 H0.1 - Shared analysis-subspace resolver

```
def resolve_analysis_subspace(
    *,
    axes: Sequence[Literal["x", "y", "z"]] | None = None,
    projection_basis: ArrayLike | None = None,
) -> AnalysisSubspace:
    ...
```

Exactly one of `axes` and `projection_basis` may be supplied. With neither, the canonical three-dimensional Cartesian basis is used. Axis labels must be unique. A general basis has shape `(d, 3)`, finite entries, rank d, and orthonormal rows within a documented tolerance.

Axis-subset projections may be formed from stored Cartesian components. A rotated projection requires the full tensor; otherwise the consumer rejects the request rather than discarding cross terms.

17.2 H0.2 - Shared semantic signature

Velocity preparation, MSD, VACF, and transport results construct the `DynamicsInputSignature` defined above from one helper. Comparison functions compare signatures before numerical fitting. The exact drift-reference atom indices and exact analyzed frame sequence are mandatory.

17.3 H0.3 - Deep immutability

All public VACF/dynamics result types use read-only owned arrays and recursively frozen metadata. Tests must attempt mutation of every array category and nested metadata category. No migration may change numerical values.

17.4 H0.4 - Validation and plateau sampling

- reject booleans for integer stride, lag, block-size, and point-count options;
- require actual booleans for boolean switches;
- centralize finite-positive scalar validation;
- require the selected GK2 plateau samples to be uniformly spaced before using an arithmetic mean; and
- continue to report no inferential standard error from serially correlated running-integral samples.

A later time-weighted plateau estimator may support irregular coordinates under a separately named method. H0 does not silently change the current estimator.

17.5 H0.5 - Implemented-path correction and compatibility

- GK1 derives dimensionality from the selected subspace;
- GK3 derives its MSD divisor from the same stored subspace;
- the legacy `dimensions` argument is deprecated and cannot be used to reinterpret a full scalar result as one- or two-dimensional;
- three-dimensional scalar and `x`/`y`/`z` calls retain their numerical results;
- legacy scalar `dimensions=1` or `2` calls raise a targeted error unless an explicit corresponding subspace is supplied; and
- result metadata records the basis, rank, labels, and source signature.

17.6 H0 acceptance tests

- reproduce every existing 3D scalar and Cartesian GK1/GK3 reference value;
- show that an anisotropic synthetic tensor gives the correct `xy`, `xz`, and rotated-subspace trace;
- reject a rotated subspace when a required source tensor is absent;
- reject equal drift-mode names with different drift-reference atoms;
- reject different trajectory slices from the same filename;
- reject different coordinate/reference-cell or velocity provenance;
- verify recursive result immutability; and
- verify strict integer/boolean validation and uniform plateau-grid enforcement.

18 GK1 - Running self-diffusion from VACF

Implementation status: implemented in `mdstats 0.19.1a0` and hardened in `mdstats 0.19.80a0`; the subspace contract below is current. See `docs/specs/analysis/vacf_transport_spec.md`.

18.1 Current function

```
def integrate_vacf_to_diffusion(
    vacf: VACFResult,
    *,
    axes: Sequence[Literal["x", "y", "z"]] | None = None,
    projection_basis: ArrayLike | None = None,
    maximum_time_ps: float | None = None,
    integration: Literal["trapezoid"] = "trapezoid",
) -> VACFDiffusionResult:
    ...
```

The compatibility layer may temporarily accept the old `component` and `dimensions` parameters, but only unambiguous 3D scalar or single-axis meanings are valid. `dimensions` is not an independent physical input.

18.2 Result contract

`VACFDiffusionResult` retains the running curve and integrand and additionally stores the resolved projection basis, optional axis labels, derived rank, and `DynamicsInputSignature`. Its arrays and metadata are deeply immutable.

18.3 Borrowed theory and numerical machinery

The diffusion coefficient follows Green-Kubo theory [5, 6]. For an explicit orthonormal basis B,

$$D_B(t) = \frac{1}{d}\int_0^t \mathrm{tr}\big[BC(\tau)B^{\mathsf{T}}\big]\,d\tau.$$

The cumulative composite trapezoidal rule is standard sampled-data quadrature, implemented through SciPy [13]. Subspace resolution, strict weighting guards, running-result schema, and separation from plateau fitting are `mdstats` design.

18.4 Numerical contract

At stored lag k,

$$D_{B,k} = \frac{1}{d}\sum_{j=0}^{k-1}\frac{C_{B,j}+C_{B,j+1}}{2}(t_{j+1}-t_j).$$

Axis subsets use the corresponding diagonal components. A rotated basis uses the stored VACF tensor. The result begins exactly at zero and remains on the source lag grid.

18.5 Inputs and constraints

- uniform per-atom weighting, or metadata proving an equivalent normalization, is required;
- mass-weighted and nonuniform explicit VACFs are rejected for self diffusion;
- the projection rank is derived from the basis;
- `maximum_time_ps` truncates at an existing lag boundary;
- negative and nonmonotonic running behavior is preserved; and
- the function returns a running diagnostic, not an automatically converged scalar coefficient.

18.6 Required tests

- all existing exponential, damped-cosine, constant, truncation, and weighting tests;
- exact axis-subset results for anisotropic synthetic data;
- rotated-basis tensor oracle;
- component additivity and 3D compatibility regression;
- rejection of the legacy full-scalar `dimensions=1/2` reinterpretation;
- signature, basis, and deep-immutability identities.

19 GK2 - Diffusion plateau estimation

Implementation status: implemented in `mdstats 0.19.4a0`; uniform selected-grid validation and deep immutability were completed in H0 (`mdstats 0.19.80a0`). See `docs/specs/analysis/diffusion_estimation_spec.md`.

19.1 Public function

```
def estimate_diffusion_plateau(
    running: VACFDiffusionResult,
    *,
```

```
    time_range_ps: tuple[float, float] | None = None,
    minimum_points: int = 8,
    slope_tolerance: float | None = None,
    method: Literal["explicit", "stable_window"] = "explicit",
) -> DiffusionEstimate:
    ...
```

19.2 Policy

- `method="explicit"` requires a user-selected interval and uses existing lag samples only;
- the selected samples must be uniformly spaced before an arithmetic mean is used;
- centered ordinary least squares reports slope, intercept, R^2, and residual diagnostics;
- an optional absolute slope tolerance records pass/fail without replacing the user's interval choice;
- adjacent running-integral samples are serially correlated, so the deterministic kernel does not fabricate an independent-sample standard error;
- `stable_window` remains unimplemented until it receives a separate algorithm and validation specification; and
- no automatic hydrodynamic or exponential tail fit is included.

`DiffusionEstimate` stores the inherited subspace and semantic signature. All arrays, diagnostics, and metadata are deeply immutable.

19.3 Required tests

- constant plateau and slowly drifting interval;
- slope-tolerance pass/fail;
- oscillatory spread without automatic acceptance;
- explicit deferral of `stable_window`;
- too-short and out-of-range rejection;
- irregular selected-grid rejection;
- exact interval, subspace, signature, and provenance preservation;
- no fabricated inferential standard error; and
- recursive immutability.

20 GK3 - MSD/VACF consistency comparison

Implementation status: implemented in `mdstats 0.19.4a0`; full semantic compatibility and subspace handling were completed in H0 (`mdstats 0.19.80a0`). See `docs/specs/analysis/diffusion_estimation_spec.md`.

20.1 Current function

```
def compare_msd_vacf_diffusion(
    msd: MSDResult,
    vacf_diffusion: DiffusionEstimate,
    *,
    msd_fit_range_ps: tuple[float, float],
) -> DiffusionComparisonResult:
    ...
```

The comparison derives the projection basis and rank from the VACF diffusion estimate. It does not accept an independent `dimensions` value.

20.2 Theory

For the same subspace B,

$$M_B(t) \sim 2dD_B t,$$

while Green-Kubo supplies the time integral of the projected VACF. Agreement is a physical and numerical consistency check, not an implementation identity.

20.3 Current H0 policy

- never recompute MSD or VACF;
- require time-averaged laboratory-frame MSD for this first comparison;
- compare complete `DynamicsInputSignature` values, including exact frame slice, measured atoms, drift-reference atoms, coordinate/reference-cell convention, velocity provenance, and projection basis;
- use stored Cartesian components for axis subsets;
- require the MSD tensor for a rotated projection;
- fit the selected projected MSD by centered ordinary least squares with an intercept;
- compute $D_B = b/(2d)$ from the derived rank;
- report signed, absolute, and symmetric relative differences; and
- preserve negative slopes and unassessed/failed plateau diagnostics as flags.

Legacy results without complete signatures are rejected by default. A separately named legacy-compatibility helper may be added only if it reports that exact identity could not be proven.

20.4 Required tests

- exact 3D and single-axis compatibility regressions;
- `xy` and rotated-subspace synthetic comparisons;
- fixed-origin and reference-cell rejection;
- mismatch rejection for frame slice, atom selection, drift-reference atoms, source identity, coordinate mode, velocity source, and projection basis;
- rotated projection without MSD tensor rejection;
- too-short fit interval and negative-slope diagnostics; and
- deep result immutability.

21 GK4 - VACF-to-MSD reconstruction consistency check

Implementation status: implemented in `mdstats 0.19.5a0`; shared subspace, semantic-signature, and deep-immutability migration were completed in H0 (`mdstats 0.19.80a0`).

21.1 Current function

```
def reconstruct_msd_from_vacf(
    vacf: VACFResult,
    *,
    axes: Sequence[Literal["x", "y", "z"]] | None = None,
    projection_basis: ArrayLike | None = None,
    maximum_time_ps: float | None = None,
    integration: Literal["trapezoid"] = "trapezoid",
) -> VACFMSDResult:
    ...
```

The legacy `component` parameter may remain temporarily as a compatibility adapter for the full scalar or one Cartesian axis. The canonical contract uses the same explicit subspace as GK1 and GK3.

21.2 Purpose and borrowed theory

The function reconstructs the finite-time projected MSD implied by a stored physical self VACF. It is a consistency diagnostic against direct position-based `compute_msd()`, not a replacement for that estimator.

Einstein's displacement representation, Green-Kubo correlation transport, and Helfand's integrated-flux formulation provide the physical lineage [5, 6, 14, 15]. For the projected scalar VACF C_B,

$$M_B(t) = 2\int_0^t (t-\tau)C_B(\tau)\,d\tau.$$

Define

$$I_0(t) = \int_0^t C_B(\tau)d\tau, \qquad I_1(t) = \int_0^t \tau C_B(\tau)d\tau.$$

Then

$$M_B(t) = 2[tI_0(t) - I_1(t)].$$

Both integrals use N2.1. The sampled two-moment $O(T)$ rearrangement, subspace resolver, immutable result schema, and provenance policy are `mdstats` design.

21.3 Current H0 result and policy

`VACFMSDResult` stores lag times, projected reconstructed MSD, the physical projected VACF, both cumulative moments, the projection basis, derived rank, and `DynamicsInputSignature`.

- require uniform or explicitly equal positive per-atom weights;
- reject mass-weighted and nonuniform explicit VACFs;
- form axis subsets from Cartesian components and rotated subspaces from the full VACF tensor;
- select `maximum_time_ps` only at an existing stored lag;
- preserve negative VACF lobes and nonmonotonic reconstructed behavior;
- store and revalidate C_B, I_0, I_1, and M_B;
- return read-only arrays and recursively frozen metadata; and
- state explicitly that finite-record agreement with direct MSD is not forced.

21.4 Required tests

- all existing constant-velocity, exponential-correlation, truncation, and weighting tests;
- 3D and Cartesian compatibility regression;
- axis-subset and rotated-tensor reconstruction;
- scalar/subspace additivity;
- direct `compute_msd()` agreement for controlled ballistic motion;
- rotated projection without tensor rejection;
- signature and deep-immutability validation.

22 DH0 - Shared displacement preparation and blocked iterator

Status: implemented in `mdstats 0.19.81a0`. See `docs/specs/analysis/_displacement_common_spec.md`.

DH0 is an `mdstats` internal architecture, not a borrowed published algorithm. Its numerical oracle is direct subtraction of validated, preprocessed positions. The implementation reproduces the existing direct time-averaged MSD within floating-point tolerance before any new observable consumes it.

22.1 D0.1 - Displacement input preparation

```
def prepare_displacement_inputs(
    collection: AtomisticFrameCollection,
    *,
    species: SpeciesSelection = None,
    atom_indices: ArrayLike | None = None,
    coordinate_mode: CoordinateMode = "laboratory",
```

```
    reference_cell: ReferenceCellInput = "initial",
    drift_mode: DriftMode | None = None,
    drift_species: SpeciesSelection = None,
    drift_atom_indices: ArrayLike | None = None,
    axes: Sequence[Literal["x", "y", "z"]] | None = None,
    projection_basis: ArrayLike | None = None,
) -> DisplacementInputBundle:
    ...
```

The bundle resolves exactly once:

- trajectory and time validation;
- canonical measured-atom selection;
- laboratory or reference-cell coordinate construction;
- the exact reference-cell matrix and policy;
- drift mode and canonical drift-reference atoms;
- analysis subspace;
- source provenance and `DynamicsInputSignature`; and
- a memory-safe position accessor or owned prepared array.

No downstream displacement observable may repeat these choices independently.

22.2 D0.2 - Blocked iterator

```
def iter_displacement_blocks(
    bundle: DisplacementInputBundle,
    lag_steps: ArrayLike,
    *,
    origin_stride: int = 1,
    atom_block_size: int | None = None,
    origin_block_size: int | None = None,
    memory_target_bytes: int | None = None,
) -> Iterator[DisplacementBlock]:
    ...
```

A block has the normative schema

```
@dataclass(frozen=True, slots=True)
class DisplacementBlock:
    lag_index: int
    lag_step: int
    lag_time_ps: float
    origin_indices: NDArray[np.int64]       # (O,)
    atom_indices: NDArray[np.int64]         # (A,)
    displacements: NDArray[np.float64]      # (O, A, d)
    n_samples: int                          # O * A
```

Iteration order is lag-major, then origin-block-major, then atom-block-major. Lag zero is permitted. Each displacement is

$$B[\mathbf{r}_i(t_0 + t) - \mathbf{r}_i(t_0)]$$

after the bundle's coordinate and drift conventions have been applied.

Both atom and origin blocking are required. Atom blocking alone does not bound memory for a very long trajectory. `memory_target_bytes`, when supplied, resolves deterministic block sizes before iteration and never changes numerical ordering. The conservative D0 work estimate is

$$b_{\text{peak}} = 8A_bO_b(9 + 2d),$$

where A_b and O_b are the resolved atom and origin block sizes and d is the subspace rank. The direct MSD path uses a 256 MiB default target and records the resolved plan in result metadata.

22.3 D0.3 - MSD migration gate

The existing direct time-averaged MSD backend is refactored to consume D0. Acceptance requires numerical identity with the pre-refactor implementation for:

- fixed and variable cells;
- laboratory and reference-cell coordinates;
- all drift modes and independent drift selections;
- scalar, Cartesian, and tensor moments;
- origin and lag stride;
- multiple atom/origin block sizes; and
- per-atom output.

The FFT MSD backend is not rewritten merely to force shared code. It retains its independent algebra and continues to agree with the D0 direct oracle. D0 focused tests cover explicit atom order, projected values, complete sample coverage, multiple block shapes, hard memory targets, drift/reference-cell preparation, and deep immutability.

23 DH1 - Self van Hove function

Status: implemented in `mdstats 0.19.82a0`. The normative module-owned specification is `docs/specs/analysis/displaceme`

23.1 Public function

```
def compute_self_van_hove(
    collection: AtomisticFrameCollection,
    *,
    species: SpeciesSelection = None,
    atom_indices: ArrayLike | None = None,
    lag_steps: ArrayLike | None = None,
    max_lag: int | None = None,
    origin_stride: int = 1,
    radial_edges: ArrayLike | None = None,
    r_max: float | None = None,
    n_bins: int = 200,
    require_complete_support: bool = False,
    axes: Sequence[Literal["x", "y", "z"]] | None = None,
    projection_basis: ArrayLike | None = None,
    coordinate_mode: CoordinateMode = "laboratory",
    reference_cell: ReferenceCellInput = "initial",
    drift_mode: DriftMode | None = None,
    drift_species: SpeciesSelection = None,
    drift_atom_indices: ArrayLike | None = None,
    atom_block_size: int | None = None,
    origin_block_size: int | None = None,
) -> SelfVanHoveResult:
    ...
```

23.2 Borrowed theory

The space-time correlation framework and self part follow van Hove [7]. The radial normalization, finite-support accounting, shared projection API, blocked iterator, and result schema are `mdstats` design.

23.3 Implemented lag and blocking contract

- `lag_steps` and `max_lag` are mutually exclusive;
- explicit lags are nonnegative, strictly increasing saved-frame integers;
- with neither lag input, D1 reports every lag through $\lfloor T/2 \rfloor$, matching the default many-origin MSD window;
- `origin_stride`, atom blocks, and origin blocks are resolved through D0; and
- D1 applies the package 256 MiB displacement-memory target and records the resolved plan in result metadata.

23.4 Histogram and support contract

- `radial_edges` and `r_max` are mutually exclusive;
- explicit edges are finite, strictly increasing, and begin at zero;
- bins are left-closed/right-open, except the final bin includes its right edge;
- if neither support input is supplied, a deterministic prepass finds the maximum observed projected displacement and constructs complete support;
- nonzero automatic support ends one representable `float64` value above the observed maximum; exactly static data use a finite $\sqrt{\epsilon_{64}}$ coordinate-scale support to avoid zero-width bins;
- with user-limited support, out-of-range samples are counted, not silently discarded or renormalized away;
- `shell_probability` is divided by the total sample count, so `shell_probability.sum(axis=1) + overflow_probability == 1`; and
- `require_complete_support=True` raises if any overflow is observed.

For bin edges r_j, r_{j+1}, shell measure is

$$\mu_j^{(1)} = 2(r_{j+1} - r_j),$$

$$\mu_j^{(2)} = \pi(r_{j+1}^2 - r_j^2),$$

$$\mu_j^{(3)} = \frac{4\pi}{3}(r_{j+1}^3 - r_j^3).$$

Thus the stored radial density satisfies the appropriate full-space normalization when overflow is zero. In one dimension the radial coordinate is $|s| \geq 0$, and the factor of two accounts for the two directions.

23.5 Result type

```
@dataclass(frozen=True, slots=True)
class SelfVanHoveResult:
    lag_steps: NDArray[np.int64]
    lag_times: NDArray[np.float64]
    radial_edges: NDArray[np.float64]
    radial_centers: NDArray[np.float64]
    shell_measure: NDArray[np.float64]
    shell_probability: NDArray[np.float64]  # (L, B)
    density: NDArray[np.float64]            # (L, B)
    counts: NDArray[np.int64]               # (L, B)
    overflow_counts: NDArray[np.int64]      # (L,)
    overflow_probability: NDArray[np.float64]
    n_samples: NDArray[np.int64]
    direct_second_moment: NDArray[np.float64]
    atom_indices: NDArray[np.int64]
    projection_basis: NDArray[np.float64]
    signature: DynamicsInputSignature
    metadata: Mapping[str, Any]
```

`captured_probability` is a derived read-only property equal to `1 - overflow_probability`. `direct_second_moment` is accumulated from all D0 samples before support filtering, so overflow never changes the MSD cross-check.

23.6 Required tests

- static atoms and deterministic translation;
- exact endpoint ownership and overflow accounting;
- isotropic Gaussian displacement in one, two, and three dimensions;
- shell-measure normalization;
- direct second moment agrees exactly with D0/MSD, while center-based histogram moments agree within documented bin error;
- complete-support prepass and strict-support rejection;
- atom/origin block invariance; and
- deep immutability and signature preservation.

24 DH2 - Non-Gaussian parameter

Status: implemented in `mdstats 0.19.83a0`.

Normative module specification:

`docs/specs/analysis/displacement_dynamics_spec.{md,pdf}`

24.1 Public function

```
def compute_non_gaussian_parameter(
    collection: AtomisticFrameCollection,
    *,
    species: SpeciesSelection = None,
    atom_indices: ArrayLike | None = None,
    max_lag: int | None = None,
    origin_stride: int = 1,
    lag_stride: int = 1,
    axes: Sequence[Literal["x", "y", "z"]] | None = None,
    projection_basis: ArrayLike | None = None,
    coordinate_mode: CoordinateMode = "laboratory",
    reference_cell: ReferenceCellInput = "initial",
    drift_mode: DriftMode | None = None,
    drift_species: SpeciesSelection = None,
    drift_atom_indices: ArrayLike | None = None,
    atom_block_size: int | None = None,
    origin_block_size: int | None = None,
) -> NonGaussianResult:
    ...
```

The non-Gaussian parameter follows the displacement-cumulant expansion; an early liquid-scattering formulation is given by Rahman, Singwi, and Sjolander [8]. For the resolved rank d,

$$\alpha_2(t) = \frac{d}{d+2}\frac{\langle r^4(t)\rangle}{\langle r^2(t)\rangle^2} - 1.$$

The projected norm $r = \|B\Delta\mathbf{r}\|$ and $d = \operatorname{rank} B$ come from the same D0 subspace. The implementation accumulates unbinned second and fourth moments in one blocked pass. It never derives moments from D1 histogram centers.

`alpha2` is `NaN` at every lag where the second moment is exactly zero, not only at lag zero. `undefined_mask` records those lags. At all other lags `alpha2` must be finite. Intermediate moment overflow is rejected.

24.2 Result type

```python
@dataclass(frozen=True, slots=True)
class NonGaussianResult:
    lag_steps: NDArray[np.int64]
    lag_times: NDArray[np.float64]
    second_moment: NDArray[np.float64]
    fourth_moment: NDArray[np.float64]
    alpha2: NDArray[np.float64]
    undefined_mask: NDArray[np.bool_]
    n_samples: NDArray[np.int64]
    atom_indices: NDArray[np.int64]
    projection_basis: NDArray[np.float64]
    signature: DynamicsInputSignature
    metadata: Mapping[str, Any]
```

The constructor validates the rank-dependent formula, exact undefined-mask semantics, moment nonnegativity, lag times, sample counts, signature identity, and recursive immutability.

24.3 Required tests

- isotropic Gaussian increments in ranks one, two, and three;
- deterministic fixed-radius motion with $\alpha_2 = -2/(d+2)$;
- a heterogeneous two-population displacement mixture;
- direct second-moment agreement with D1 and D0/MSD;
- later-lag exact zero moments;
- rotated projection bases and rank-correct prefactors;
- atom/origin block invariance and exact sample counts;
- signature preservation, constructor invariants, and deep immutability.

25 DH3 - Self-intermediate scattering function

Status: implemented in `mdstats 0.19.84a0`.

25.1 Public function

```python
def compute_self_intermediate_scattering(
    collection: AtomisticFrameCollection,
    *,
    q_vectors: ArrayLike | None = None,
    q_magnitudes: ArrayLike | None = None,
    isotropic: bool = True,
    species: SpeciesSelection = None,
    atom_indices: ArrayLike | None = None,
    max_lag: int | None = None,
    origin_stride: int = 1,
    lag_stride: int = 1,
    axes: Sequence[Literal["x", "y", "z"]] | None = None,
    projection_basis: ArrayLike | None = None,
    coordinate_mode: CoordinateMode = "laboratory",
    reference_cell: ReferenceCellInput = "initial",
    drift_mode: DriftMode | None = None,
    drift_species: SpeciesSelection = None,
    drift_atom_indices: ArrayLike | None = None,
    atom_block_size: int | None = None,
    origin_block_size: int | None = None,
) -> SelfIntermediateScatteringResult:
```

...

The self-intermediate scattering formalism follows van Hove [7] and Vineyard [9]. D3 evaluates it from the direct D0 displacement samples rather than from a binned D1 histogram.

25.2 Explicit-vector mode

For `isotropic=False`, callers supply Cartesian wavevectors with shape `(Q,3)` and units of inverse angstrom. D3 computes

$$F_s(\mathbf{q}, t) = \left\langle e^{i\mathbf{q}\cdot\Delta\mathbf{r}(t)} \right\rangle$$

directly and returns complex values. With D0 row basis B, admissible vector coordinates are $\mathbf{q}_d = \mathbf{q}B^{\mathsf{T}}$. The residual outside the subspace must satisfy

$$\|\mathbf{q} - \mathbf{q}B^{\mathsf{T}}B\|_2 \leq 10^{-10} \max(1, \|\mathbf{q}\|_2).$$

An out-of-subspace component is rejected instead of silently discarded. Input ordering and duplicates are preserved. Requested $\pm\mathbf{q}$ pairs obey the expected complex-conjugation identity.

25.3 Isotropic-magnitude mode

For `isotropic=True`, callers supply finite nonnegative q magnitudes in inverse angstrom. D3 applies the angular kernel for the resolved subspace rank:

$$K_1(qr) = \cos(qr), \qquad K_2(qr) = J_0(qr), \qquad K_3(qr) = j_0(qr) = \frac{\sin(qr)}{qr}.$$

The two- and three-dimensional special functions use SciPy [13]. The q-zero value and every lag-zero value are set exactly to one after accumulation.

`q_vectors` and `q_magnitudes` are mutually exclusive. `isotropic=True` requires magnitudes; `isotropic=False` requires vectors. No implicit mode inference is performed.

25.4 Result and blocked evaluation

`SelfIntermediateScatteringResult` stores lag axes, the original q input, projected q coordinates for vector mode, real or complex values, exact sample counts, measured atoms, projection basis, complete dynamics signature, and recursively immutable metadata. All arrays are owned and read-only.

D3 reuses the D0 lag/origin/atom block plan. Within each displacement block, q values are processed in contiguous private chunks under a fixed transient-work target. This bounds temporary phase or kernel arrays without changing the explicit `(lag,q)` output, q order, duplicate retention, or sample membership.

Required validation includes zero lag and zero q, ballistic phase, Gaussian increments in all supported ranks, direct SciPy kernel checks, fine D1 van-Hove-transform comparison, conjugation identities, rotated subspaces, q-order preservation, block invariance, constructor invariants, and deep immutability.

26 C0 - Collective-current contract closure

Status: implemented in `mdstats 0.19.85a0` as the contract boundary for C1.

The Green-Kubo conductivity relation is established theory [5, 6]. C0 resolves package-level ambiguities that the physical formula alone does not settle.

26.1 Charge input

The first implementation accepts exactly one of:

- a per-atom charge array of shape `(N,)`; or
- `species_charges: Mapping[str, float]` keyed by exact chemical symbols.

Integer-key mappings are not accepted because an integer could mean an atom index or atomic number. A symbol mapping must cover every element present in the collection and may not contain unused symbols. Charge values are in units of the elementary charge e. Resolved per-atom charges are stored explicitly. Atoms with exactly zero charge are retained in that array but excluded from the current-carrying atom population.

Periodic conductivity requires charge neutrality by default:

$$\left|\sum_i q_i\right| \leq \varepsilon_Q.$$

A non-neutral override, if added later, must be explicit, must store the net charge, and must warn that uniform drift changes the current by $Q_{\mathrm{tot}}\mathbf{v}_{\mathrm{drift}}$. The implemented C1 release rejects non-neutral systems rather than providing that override.

26.2 Species-group partition

When group decomposition is requested, groups must form a disjoint, exhaustive partition of the current-carrying atoms. This makes

$$\mathbf{J}_{\mathrm{tot}} = \sum_a \mathbf{J}_a$$

an exact result identity. Partial or overlapping diagnostic groupings require a later separately named API and do not claim the decomposition identity.

26.3 Correlation ordering

Positive-lag cross correlations are ordered:

$$C_{ab}(t) = \langle \mathbf{J}_a(0) \cdot \mathbf{J}_b(t) \rangle.$$

They need not satisfy $C_{ab}(t) = C_{ba}(t)$ at positive lag. CC2 stores the full ordered matrix. Any symmetrized representation is derived explicitly and never replaces the ordered source data.

26.4 Volume and thermodynamic provenance

`ChargeCurrentResult` and `CurrentCorrelationResult` carry the complete instantaneous volume series, periodic-axis flags, fixed/variable **cell-matrix** provenance, and the complete dynamics signature. A trajectory whose determinant is constant but whose cell matrix changes is variable-cell. CC3 accepts only a fixed physical volume in its first release. A scalar argument cannot be used to hide a variable-cell trajectory.

26.5 Comparison provenance

CC4 requires matching trajectory signature, species partition, temperature, volume, charges, and diffusion subspace. Named ratios with conflicting conventions are avoided; the result reports both `sigma_collective/sigma_NE` and its inverse explicitly.

27 CC1 - Charge-current construction

Status: implemented in `mdstats 0.19.85a0`.

27.1 Public function

```
def compute_charge_current(
    collection: AtomisticFrameCollection,
    *,
    charges: ArrayLike | None = None,
    species_charges: Mapping[str, float] | None = None,
    species_groups: Mapping[str, SpeciesSelection] | None = None,
    drift_mode: DriftMode | None = None,
    drift_species: SpeciesSelection = None,
    drift_atom_indices: ArrayLike | None = None,
    neutrality_tolerance_e: float = 1.0e-12,
) -> ChargeCurrentResult:
    ...
```

For resolved charges q_i and velocities $\mathbf{v}_i(t)$,

$$\mathbf{J}_q(t) = \sum_i q_i \mathbf{v}_i(t).$$

The result stores total and optional group currents in e angstrom/ps, resolved charges, total charge, group partition, cell-volume provenance, and `DynamicsInputSignature`. SI conversion belongs to CC3.

Required tests cover a neutral rigid translation, single-particle algebra under a deliberately neutral paired system, exact group sum, array/species-map agreement, missing/ambiguous charges, non-neutral rejection, overlapping or incomplete groups, exact drift-reference provenance, and deep immutability.

28 CC2 - Current autocorrelation and ordered cross-correlation

Status: implemented in `mdstats 0.19.85a0` as the second half of release stage C1.

28.1 Public function

```
def compute_current_correlation(
    current: ChargeCurrentResult,
    *,
    max_lag: int | None = None,
    origin_stride: int = 1,
    lag_stride: int = 1,
    compute_tensor: bool = True,
    backend: Backend = "auto",
) -> CurrentCorrelationResult:
    ...
```

If groups are present, store the ordered matrix

$$C_{ab}(t) = \langle \mathbf{J}_a(0) \cdot \mathbf{J}_b(t) \rangle.$$

The total correlation must equal the sum over every ordered group pair. Tensor trace and direct/FFT identities follow the same positive-lag conventions as the VACF machinery, but cross-particle terms are intentional. The implemented estimator uses the raw resolved current: it does not subtract a time mean, detrend, smooth, or symmetrize. Drift removal must be selected when constructing the current.

Required tests include direct/FFT agreement, ordered nonsymmetric positive-lag cross terms, exact group-pair sum, tensor trace identities, zero current, strict boolean/integer validation, signature preservation, and deep immutability.

29 CC3 - Ionic conductivity integration

Status: implemented in `mdstats 0.19.86a0`.

29.1 Public function

```
def integrate_ionic_conductivity(
    correlation: CurrentCorrelationResult,
    *,
    temperature_k: float,
    volume_a3: float | None = None,
    maximum_time_ps: float | None = None,
) -> IonicConductivityResult:
    ...
```

For the total current in an isotropic three-dimensional, fixed-volume system,

$$\sigma(t) = \frac{1}{3Vk_{\mathrm{B}}T}\int_0^t \langle \mathbf{J}_q(0)\cdot\mathbf{J}_q(\tau)\rangle d\tau.$$

This is the Green-Kubo conductivity relation [5, 6]. The implemented contract is explicitly three-dimensional and isotropic. It requires all periodic-axis flags, rejects variable full-cell-matrix provenance before considering `volume_a3`, and treats an explicit volume as a consistency assertion rather than an unrelated replacement.

C1 correlations have units $e^2\,\mathrm{Angstrom}^2/\mathrm{ps}^2$. After time integration in picoseconds, C2 applies

$$\frac{e^2 10^{22}}{3Vk_{\mathrm{B}}T}$$

to obtain siemens per meter. The elementary charge and Boltzmann constant are recorded in result metadata. The result retains the total and ordered group-pair correlation samples, cumulative integrals, and running SI conductivities. Its constructor independently rechecks total and group quadrature, SI conversion, charge neutrality, exact partition identity, fixed volume, periodicity, and signature consistency.

The integration does not smooth, extrapolate, fit a tail, or choose a plateau.

Required tests cover sampled analytic correlations, exact SI conversion, ordered group-pair quadrature and summation, inverse volume and temperature scaling, inconsistent-volume rejection, partial-periodic and variable-cell rejection, truncation, constructor invariants, signature preservation, and deep immutability.

30 CC3b - Explicit conductivity plateau

Status: implemented in `mdstats 0.19.86a0`.

```
def estimate_ionic_conductivity_plateau(
    running: IonicConductivityResult,
    *,
    time_range_ps: tuple[float, float],
    minimum_points: int = 8,
    slope_tolerance_s_per_m_ps: float | None = None,
) -> IonicConductivityEstimate:
    ...
```

The estimate is the arithmetic mean of the running conductivity over one explicit uniformly sampled interval. It records centered linear-fit slope, intercept, residual, range, span, endpoint drift, and optional slope-stability status. It does not infer an automatic window and does not report a naive independent-sample standard error from one serially correlated running curve. The same stored indices are averaged for every ordered group-pair contribution, whose interval means must sum to the total estimate.

31 CC4 - Nernst-Einstein comparison

Status: implemented in `mdstats 0.19.86a0`.

31.1 Public function

```
def compute_nernst_einstein_comparison(
    conductivity: IonicConductivityEstimate,
    species_diffusion: Mapping[str, DiffusionEstimate],
    *,
    temperature_k: float | None = None,
    volume_a3: float | None = None,
) -> NernstEinsteinComparisonResult:
    ...
```

Species counts and charges are derived from the conductivity/current provenance, not supplied a second time. Optional temperature and volume arguments are consistency assertions against the stored state. Each exact current group must have one uniform nonzero charge and one compatible full-three-dimensional `DiffusionEstimate`. Compatibility requires the same trajectory fingerprint, frames, times, sampling, source identity, drift mode, and drift-reference atoms, while allowing the expected displacement-versus-velocity provenance fields to differ.

The independent-particle estimate is

$$\sigma_{\mathrm{NE}} = \frac{e^2}{V k_{\mathrm{B}} T} \sum_a N_a z_a^2 D_a.$$

The result reports both conductivities, `collective - Nernst-Einstein`, the absolute difference, both directional ratios, explicit ratio-defined flags, per-group contributions, and the summed off-diagonal ordered group-pair conductivity. A zero denominator produces `NaN` and a false flag. The API does not assign a universal Haven-ratio name because reciprocal conventions coexist.

Required tests cover an independent-particle synthetic limit, controlled collective enhancement and suppression, exact species contributions, every selection/trajectory/frame/drift/subspace/charge/state mismatch, zero-denominator ratio policy, constructor invariants, and deep immutability.

32 SD1-SD3 - Topology-coupled site dynamics

These functions are deferred until ring and site topology are stable. They are included here because they are the most system-specific continuation for LTA cation dynamics.

32.1 Proposed functions

```
def assign_site_trajectory(...) -> SiteTrajectoryResult:
    ...

def compute_site_residence(...) -> SiteResidenceResult:
    ...

def compute_hopping_statistics(...) -> HoppingStatisticsResult:
    ...
```

Expected outputs include:

- site occupancy versus time;
- continuous and intermittent residence correlations;
- waiting-time distributions;
- transition counts and transition matrices;
- portal-crossing counts;

- jump lengths, durations, and recrossing diagnostics.

The exact residence and recrossing definitions must receive their own source audit before implementation. No historical attribution is assigned in this roadmap because the final estimator has not yet been selected.

33 Advanced branches deliberately deferred

33.1 Dynamic structure factor

A later function may compute collective intermediate scattering and $S(q, \omega)$ from density correlations. Its theoretical foundation is the van Hove framework [7]. It is deferred until the self-intermediate scattering API is stable.

33.2 Two-phase thermodynamics

The 2PT method uses a VACF-derived density of states and partitions it into solid-like and gas-like contributions to estimate liquid thermodynamics [10]. Later reanalysis showed that implementation details and component partitioning can materially affect predicted entropy [11]. For that reason, 2PT is not part of the first VACF extension and must receive a separate theory and validation stage.

33.3 Infrared and Raman spectra

Infrared analysis requires dipole or charge-current information. Raman analysis requires polarizability information. Neither should be represented as an option of `compute_vacf_spectrum`.

33.4 Mode-projected phonon analysis

Wavevector- or eigenvector-projected phonon spectra require a reference structure, atom mapping, phase factors, and usually harmonic eigenvectors. They belong in a later lattice-dynamics module rather than the generic VACF branch.

34 Shared numerical policies

34.1 Time sampling

All first-generation spectral functions require a strictly increasing uniform time grid. N2.1 itself may support arbitrary monotonic coordinates, but the physical VACF and Welch APIs retain the stricter uniform-sampling contract. Resampling and nonuniform Fourier methods are deferred.

34.2 Analysis subspaces

Every scalar transport or displacement observable stores an explicit orthonormal projection basis. The rank of that basis is the dimensionality. Canonical axis subsets may use component arrays; rotated subspaces require full tensors where second moments or correlations are reconstructed. No API accepts a dimensional divisor that is independent of the data projection.

34.3 Canonical frequency and units

The stored frequency is cycles per ps, numerically THz. Derived axes use `scipy.constants` [13]:

$$\omega = 2\pi f, \qquad \tilde{\nu} = \frac{f}{c}, \qquad E = hf.$$

For sample spacing Δt and FFT length N_{FFT},

$$\Delta f = \frac{1}{N_{\mathrm{FFT}} \Delta t}, \qquad f_{\mathrm{Nyquist}} = \frac{1}{2\Delta t}.$$

A separate stored spectrum is not created for angular frequency; changing the abscissa does not silently change the density ordinate by a Jacobian.

34.4 One-sided density convention

All first-generation real-signal spectra use a one-sided density. Interior positive-frequency bins are doubled; DC and an even-length Nyquist bin are not. Result metadata always records:

```
{
    "spectral_sidedness": "one_sided",
    "spectral_scaling": "density",
    "frequency_convention": "cycles_per_ps",
}
```

34.5 Reported versus biased VACF

The transform function exposes both estimators. The package does not label one universally superior:

- `reported` preserves the displayed all-origin VACF but may yield negative spectral lobes under finite noisy truncation;
- `biased` applies origin-count weighting and is the natural comparison to a full-record periodogram.

The selected form is stored in the result.

34.6 Zero padding

Use `next_fast_len` only after satisfying the two-sided embedding lower bound. Padding refines the sampled grid and plotting interpolation but does not narrow the physical resolution set by the usable time record. Total one-sided bin weight must remain invariant within roundoff.

34.7 Windows

Every window is named and stored in metadata. The two window contexts are mathematically distinct:

- VACF lag taper: multiply the positive-lag correlation by a centered half-window satisfying $w_0 = 1$;
- Welch segment window: multiply each time segment and divide periodogram power by $f_s \sum_n w_n^2$.

An ordinary full Hann array is never applied directly to a positive-lag VACF. Window leakage and resolution tradeoffs follow Harris [12].

34.8 Detrending and drift removal

Physical framewise drift removal belongs to the shared velocity-input layer. Welch segment detrending is a separate signal-processing choice. It defaults to `none` and must be explicit because constant detrending changes the low-frequency spectrum.

34.9 Spectral integration

VDOS normalization uses the discrete uniform-bin measure $\Delta f \sum_m P_m$. Trapezoidal quadrature is reserved for sampled functions such as Green-Kubo running integrals. These operations must not share a generic unnamed `integrate()` helper.

34.10 Plateau interval measure

The explicit GK2 plateau estimator uses an arithmetic mean only on a uniformly spaced selected lag grid. Irregular selected coordinates are rejected. A future time-weighted estimator must use a different method name and specification.

34.11 Negative values

- Welch scalar and diagonal periodograms must be nonnegative up to roundoff.
- A transformed reported VACF may contain larger negative lobes and preserves them by default.
- VDOS normalization rejects material negative weight.
- No implementation clips off-diagonal complex tensor elements.

34.12 Precision and accumulation

- public numerical results use `float64` or `complex128`;
- segment, atom-block, and origin-block accumulations use deterministic order;
- large sums may adopt compensated summation only after a benchmark and separate numerical specification; and
- internal work arrays remain mutable, while public result arrays are owned and read-only.

34.13 Result immutability

Every public result recursively freezes metadata and marks all stored arrays read-only after validation. Nested lists, dictionaries, and arrays may not provide a mutation backdoor. Result constructors copy inputs when exclusive ownership cannot be proven.

34.14 Strict option validation

Shared validators reject booleans where integers are required, reject integers where booleans are required, require finite positive physical scalars, and apply one consistent rule to stride, lag, block-size, and point-count arguments.

34.15 Stationarity

Spectral and Green-Kubo interpretations require a stationary production segment. The package may warn about obvious drift but cannot prove stationarity. Users must be able to slice equilibration and production intervals explicitly.

34.16 Uncertainty

Uncertainty estimation is not hidden inside deterministic kernels. Block analysis, segment variance, or bootstrap procedures are later explicit stages.

34.17 Determinism

Given identical arrays and parameters, results must be bitwise deterministic where NumPy/SciPy permit it. Atom, component, segment, group, and frequency ordering are stable.

35 Shared testing strategy

Every unit includes analytic, direct-oracle, cross-module, invalid-input, and memory/determinism tests.

35.1 Analytic synthetic tests

Use constant, sinusoidal, damped-harmonic, exponential-correlation, white-noise, Gaussian-displacement, ballistic, and deterministic-hopping data. Each synthetic case states which quantity is analytic and which is only statistically expected.

35.2 Independent numerical oracles

- N1.5 and VS1: direct $O(LF)$ Fourier summation;
- VS2: hand periodogram plus `scipy.signal.welch/csd` for small cases;
- N2.1 and GK1: direct cumulative trapezoid loop;

- VS3: explicit uniform-bin sums;
- blocked kernels: unblocked small-array implementation.

An optimized helper must never test itself through the same algebraic path.

35.3 Spectral identities

Required examples include:

- scalar spectrum equals component sum;
- tensor diagonal equals components and tensor is Hermitian;
- one-sided bin area reproduces $C(0)$ for applicable synthetic inputs;
- zero padding preserves total bin weight;
- `biased` origin weighting is exact;
- Welch density satisfies a Parseval-style energy check;
- VS1-biased and VS2 agree within expected finite-record tolerance.

35.4 Dynamical cross-module identities

- VDOS projections reproduce total VDOS;
- van Hove second moment agrees with direct MSD;
- Gaussian van Hove agrees with self-intermediate scattering;
- VACF and MSD diffusion agree for a well-sampled diffusive process;
- reconstructed VACF-MSD agrees with direct MSD in controlled synthetic data;
- species current correlations sum to total current correlation;
- projected VACF and projected MSD use the same explicit subspace;
- van Hove probability plus overflow probability equals one;
- all cross-module comparisons reject semantic-signature mismatches.

35.5 Invalid-input tests

Reject ensembles, missing velocities, nonuniform spectral times, incompatible weights, an independently supplied dimensional divisor, malformed or rank-deficient projection bases, empty selections, malformed windows, impossible segment overlap, unsupported padding, booleans passed as integer controls, missing or ambiguous charges, non-neutral periodic current systems, invalid group partitions, and unsupported variable-volume transport.

35.6 Memory and determinism tests

- force multiple atom block sizes and compare exactly or within a strict floating-point tolerance;
- verify that per-atom output selection does not change total accumulation;
- ensure no cross-atom spectral products enter self spectra;
- force independent atom and origin block sizes for displacement kernels;
- attempt mutation of every public array and nested metadata category;
- compare exact semantic signatures across compatible and incompatible slices;
- repeat seeded calculations and verify stable ordering and metadata.

36 Documentation and citation requirements

For every stage:

1. write the Markdown specification before implementation;
2. generate and inspect the PDF specification;
3. identify borrowed theory in the specification;
4. add concise source comments near the corresponding implementation;
5. state which parts are mdstats-specific;
6. add the function to package exports only after tests pass;
7. update the changelog and relevant architecture document;

8. run the full test suite before packaging.

Recommended comment style:

```
# The PSD/autocorrelation relation follows the Wiener-Khinchin theorem
# [Wiener 1930; Khintchine 1934]. The positive-lag tensor reconstruction and
# result normalization below are mdstats-specific implementation choices.

# Welch averaging follows P. D. Welch, IEEE Trans. Audio Electroacoust.
# 15, 70-73 (1967), DOI: 10.1109/TAU.1967.1161901. Atom blocking and
# self-only multi-atom aggregation are mdstats adaptations.

# The transport integral is a Green-Kubo relation [Green 1954; Kubo 1957].
# This function intentionally returns a running integral; plateau selection is
# delegated to a separate mdstats result-analysis function.
```

37 Release sequence

The revised release sequence is:

Release stage	Content	Status
N1-V5	Spectral kernels, VACF spectrum, Green-Kubo, VDOS, plotting, reconstruction, shared velocity preparation, Welch	implemented
H0	Subspace correction, semantic signature, deep immutability, strict validation, plateau-grid policy	implemented in 0.19.80a0
D0	Shared displacement preparation, atom/origin blocking, direct-MSD migration	implemented in 0.19.81a0
D1	Self van Hove with explicit overflow accounting	implemented in 0.19.82a0
D2	Non-Gaussian parameter on the shared subspace	implemented in 0.19.83a0
D3	Self-intermediate scattering with dimension-correct isotropic kernels	implemented in 0.19.84a0
C0	Charge/current contract closure	implemented in 0.19.85a0
C1	Charge current and ordered current correlations	implemented in 0.19.85a0
C2	Conductivity, explicit plateau estimation, and Nernst-Einstein comparison	implemented in 0.19.86a0
S1	Topology-coupled residence and hopping analyses	deferred until stable topology

Each stage is independently usable. No stage exposes a placeholder public function. The package version advances only after code, tests, Markdown/PDF specifications, changelog, exports, and architecture documentation agree.

38 Decision summary

The completed N1-D0 branch is now the numerical and semantic foundation. H0 closed the three defects that would otherwise have propagated into the displacement branch:

1. dimensionality is now inseparable from the physical projection;

2. MSD/VACF comparisons reject mismatched trajectory slices or drift-reference populations through complete signatures; and
3. new result objects inherit a tested deep-immutability contract.

D0 now centralizes displacement preparation and blocks both origins and atoms. The direct time-averaged MSD is the validated migration oracle. D1 consumes the same engine and adds explicit finite-support, overflow, shell-measure, captured-probability, and unbinned second-moment contracts. D2 reuses the same D0 samples for second and fourth moments with an explicit undefined-mask contract. D3 adds direct isotropic and directional self-intermediate scattering with dimension-correct kernels and exact q-space provenance.

C0-C1 resolve charges, enforce default neutrality, construct exact group partitions, retain fixed/variable-cell provenance, and compute total plus ordered positive-lag current correlations with direct/FFT agreement. C2 now applies the established Green-Kubo conductivity relation with exact SI conversion, retains ordered group-pair contributions, adds explicit plateau estimation, and performs a provenance-compatible Nernst-Einstein comparison. The next system-specific objective is topology-coupled residence and hopping after site topology is stable.

This order preserves all valid implemented results, repairs the public boundary before reuse, and keeps every new scientific observable independently testable and explicitly attributable.

39 References

[1] N. Wiener, "Generalized Harmonic Analysis," *Acta Mathematica* **55**, 117-258 (1930). DOI: 10.1007/BF02546511.

[2] A. Khintchine, "Korrelationstheorie der stationaren stochastischen Prozesse," *Mathematische Annalen* **109**, 604-615 (1934). DOI: 10.1007/BF01449156.

[3] P. D. Welch, "The Use of Fast Fourier Transform for the Estimation of Power Spectra: A Method Based on Time Averaging over Short, Modified Periodograms," *IEEE Transactions on Audio and Electroacoustics* **15**, 70-73 (1967). DOI: 10.1109/TAU.1967.1161901.

[4] A. Rahman, "Correlations in the Motion of Atoms in Liquid Argon," *Physical Review* **136**, A405-A411 (1964). DOI: 10.1103/PhysRev.136.A405.

[5] M. S. Green, "Markoff Random Processes and the Statistical Mechanics of Time-Dependent Phenomena. II. Irreversible Processes in Fluids," *Journal of Chemical Physics* **22**, 398-413 (1954). DOI: 10.1063/1.1740082.

[6] R. Kubo, "Statistical-Mechanical Theory of Irreversible Processes. I. General Theory and Simple Applications to Magnetic and Conduction Problems," *Journal of the Physical Society of Japan* **12**, 570-586 (1957). DOI: 10.1143/JPSJ.12.570.

[7] L. Van Hove, "Correlations in Space and Time and Born Approximation Scattering in Systems of Interacting Particles," *Physical Review* **95**, 249-262 (1954). DOI: 10.1103/PhysRev.95.249.

[8] A. Rahman, K. S. Singwi, and A. Sjolander, "Theory of Slow Neutron Scattering by Liquids. I," *Physical Review* **126**, 986-996 (1962). DOI: 10.1103/PhysRev.126.986.

[9] G. H. Vineyard, "Scattering of Slow Neutrons by a Liquid," *Physical Review* **110**, 999-1010 (1958). DOI: 10.1103/PhysRev.110.999.

[10] S.-T. Lin, M. Blanco, and W. A. Goddard III, "The Two-Phase Model for Calculating Thermodynamic Properties of Liquids from Molecular Dynamics: Validation for the Phase Diagram of Lennard-Jones Fluids," *Journal of Chemical Physics* **119**, 11792-11805 (2003). DOI: 10.1063/1.1624057.

[11] T. Sun, J. Xian, H. Zhang, Z. Zhang, and Y. Zhang, "Two-Phase Thermodynamic Model for Computing Entropies of Liquids Reanalyzed," *Journal of Chemical Physics* **147**, 194505 (2017). DOI: 10.1063/1.5001798.

[12] F. J. Harris, "On the Use of Windows for Harmonic Analysis with the Discrete Fourier Transform," *Proceedings of the IEEE* **66**, 51-83 (1978). DOI: 10.1109/PROC.1978.10837.

[13] P. Virtanen, R. Gommers, T. E. Oliphant, et al., "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods* **17**, 261-272 (2020). DOI: 10.1038/s41592-019-0686-2.

[14] A. Einstein, "Uber die von der molekularkinetischen Theorie der Warme geforderte Bewegung von in ruhenden Flussigkeiten suspendierten Teilchen," *Annalen der Physik* **322**, 549-560 (1905). DOI: 10.1002/andp.19053220806.

[15] E. Helfand, "Transport Coefficients from Dissipation in a Canonical Ensemble," *Physical Review* **119**, 1-9 (1960). DOI: 10.1103/PhysRev.119.1.

40 MLFF validation integration boundary

This architecture owns all numerical and physical definitions for MSD, VACF, velocity spectra, VDOS, Green-Kubo self diffusion, VACF/MSD consistency, self van Hove functions, non-Gaussian parameters, self-intermediate scattering, charge-current correlations, ionic conductivity, and Nernst-Einstein comparison.

`mdstats.training_data` may request these observables through standardized analysis calls, pair reference and MLFF executions, and record the protocol. It must not reproduce FFTs, correlation estimators, integration rules, projection semantics, conductivity prefactors, plateau estimators, or stationarity rules.

Revision 0.20.44a0 registers the implemented public functions under stable call IDs through the analysis-owned observable-validation facade. This is dispatch only; the owner functions and native result objects are unchanged.

The following physically important additions remain outside the implemented boundary and require their own analysis work before MLFF may expose them:

- dynamic structure factor;
- two-phase thermodynamics;
- infrared and Raman spectra;
- mode-projected phonon analysis;
- shear viscosity from stress autocorrelation.

Viscosity is not a VACF specialization. It requires a stress/flux-correlation contract, stress provenance, tensor-component policy, finite-size and plateau analysis, and should receive a separate transport specification or manual.

40.1 Stress-correlation viscosity boundary

The common correlation, integration, block-uncertainty, and plateau utilities may be shared with stress-based transport. Shear and bulk viscosity themselves are owned by `thermomechanical_energetic_validation_architecture.{md,p` because their primary microscopic observable is the stress tensor. This manual continues to own velocity-, displacement-, and charge-current-based dynamics and transport. No duplicate correlation engine is permitted.